หน้าหลัก
1 Data Science คืออะไร?
2 การตั้งคำถามและกำหนดปัญหา
3 Python สำหรับ Data Science
4 การเก็บข้อมูลและ Pandas
5 การทำความสะอาดและแปลงข้อมูล
6 สถิติเชิงพรรณนา EDA และ Data Visual
7 แนะนำ Machine Learning
8 Regression — การทำนายค่าต่อเนื่อง
9 Classification — การจำแนกประเภท
  ปัญหา Classification
  Decision Tree
  k-Nearest Neighbors
  Classification Metrics
  การเปรียบเทียบและเลือก Model
  Logistic Regression
  สรุปท้ายบทที่ 9
  เอกสารอ้างอิงประจำบทที่ 9
  แบบฝึกหัดท้ายบทที่ 9
10 Ensemble Methods
11 Clustering — การจัดกลุ่มข้อมูล
12 Model Validation และ Selection
13 Data Storytelling และการเขียนรายงาน
14 โครงงาน Data Science: End-to-End Pr
อภิธานศัพท์
ดรรชนี

บทที่ 9: Classification — การจำแนกประเภท

ภาค 3: Machine Learning **CLO:** CLO2, CLO3 **สัปดาห์:** 11

วัตถุประสงค์การเรียนรู้

เมื่อศึกษาบทนี้จบแล้ว นักศึกษาจะสามารถ:

1. อธิบายหลักการ Classification และความแตกต่างจาก Regression ได้
2. สร้างโมเดล Decision Tree และอธิบายโครงสร้างได้
3. สร้างโมเดล k-Nearest Neighbors (KNN) และเลือก k ที่เหมาะสมได้
4. อ่านและตีความ Confusion Matrix ได้
5. คำนวณ Precision, Recall, F1-Score และเลือก Metric ที่เหมาะสมกับบริบทได้
6. สร้างโมเดล Logistic Regression และเชื่อมโยงกับ Regression ได้

ปัญหา Classification

การจำแนกประเภท (Classification) เป็นหนึ่งในงานหลักที่สำคัญที่สุดของแขนงการเรียนรู้ของเครื่องแบบมีผู้สอน (Supervised Machine Learning) โดยมีวัตถุประสงค์เพื่อสร้างโมเดลที่สามารถเรียนรู้ความสัมพันธ์ระหว่างตัวแปรอิสระ (Independent Variables หรือ Features) และตัวแปรตาม (Dependent Variable หรือ Target) เพื่อทำนายหมวดหมู่หรือกลุ่ม (Category/Class) ของข้อมูลใหม่ที่ไม่เคยพบเห็นมาก่อนได้อย่างถูกต้อง (James et al., 2023)

ในทางคณิตศาสตร์ ปัญหา Classification คือการประมาณค่าฟังก์ชัน f ที่ทำการจับคู่ (Mapping) ข้อมูลนำเข้า X ไปยังผลลัพธ์ Y โดยที่ Y เป็นค่าแบบไม่ต่อเนื่อง (Discrete values) ซึ่งเป็นตัวแทนของกลุ่มหรือหมวดหมู่ต่างๆ สามารถเขียนให้อยู่ในรูปสมการได้ดังนี้:

$$Y = f(X)$$

โดยที่ $Y \in \{C_1, C_2, \ldots, C_k\}$ เมื่อ k คือจำนวนหมวดหมู่ทั้งหมดที่เป็นไปได้ และ C คือป้ายกำกับ (Label) ของหมวดหมู่นั้นๆ

เมื่อใดที่ควรใช้ปัญหา Classification

หลักเกณฑ์สำคัญที่สุดในการพิจารณาว่าปัญหาที่กำลังศึกษานั้นจัดอยู่ในกลุ่ม Classification หรือไม่ คือการพิจารณาชนิดของข้อมูลของ **ตัวแปรตาม (Target Variable)** หากตัวแปรตามเป็นข้อมูลเชิงคุณภาพ (Qualitative Data) หรือข้อมูลเชิงตัวหมวดหมู่ (Categorical Data) ปัญหานั้นจะจัดเป็นปัญหา Classification ทันที

เพื่อความเข้าใจที่ชัดเจนยิ่งขึ้น พิจารณาความแตกต่างระหว่างปัญหา Classification และปัญหา Regression ซึ่งเป็น Supervised Learning ทั้งคู่ แต่มีจุดประสงค์การใช้งานที่แตกต่างกันอย่างสิ้นเชิง ดังตารางที่ 9.1

ตารางที่ 9.1 การเปรียบเทียบระหว่างปัญหา Classification และ Regression

คุณลักษณะ	Classification (การจำแนกประเภท)	Regression (การวิเคราะห์การถดถอย)
ลักษณะของตัวแปรตาม (Target)	ไม่ต่อเนื่อง (Discrete), เป็นหมวดหมู่ (Categorical)	ต่อเนื่อง (Continuous), เป็นตัวเลข (Numerical)
ผลลัพธ์ของการทำนาย	หมวดหมู่ (Class) หรือ ความน่าจะเป็นของหมวดหมู่	ค่าตัวเลขที่จับต้องได้ (Value)
ตัวอย่างคำถาม	ลูกค้าคนนี้จะยกเลิกบริการหรือไม่? (ใช่/ไม่ใช่)	ลูกค้าคนนี้จะใช้จ่ายเงินเท่าใด? (บาท)
ตัวอย่างบริบท	ภาพนี้คือเซลล์มะเร็งชนิดใด? (เนื้อร้าย/เนื้อดี)	บ้านหลังนี้ควรมีราคาเท่าใด? (บาท)
การประเมินผล (Metrics)	Accuracy, Precision, Recall, F1-Score	MSE, MAE, R-Squared

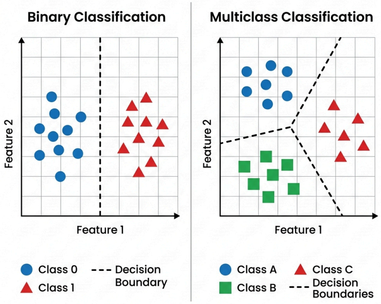
ประเภทของปัญหา Classification

ปัญหา Classification สามารถแบ่งออกได้เป็น 3 ประเภทหลัก ตามลักษณะและจำนวนของหมวดหมู่ที่เป็นผลลัพธ์ ดังนี้

1. Binary Classification (การจำแนกประเภทแบบทวิวิถี) เป็นรูปแบบที่พื้นฐานและพบได้บ่อยที่สุด โดยตัวแปรตามจะมีเพียง 2 หมวดหมู่ที่เป็นไปได้เท่านั้น (Mutually exclusive) มักถูกแทนค่าด้วย 0 และ 1 หรือ Positive และ Negative * *ตัวอย่าง:* การตรวจจับอีเมลขยะ (Spam = 1, ไม่ใช่ Spam = 0), การทำนายการเกิดโรค (เป็นโรค = 1, ไม่เป็นโรค = 0)

2. Multiclass Classification (การจำแนกประเภทแบบหลายหมวดหมู่) เป็นปัญหาที่ตัวแปรตามมีมากกว่า 2 หมวดหมู่ โดยที่ข้อมูลหนึ่งตัวอย่างจะถูกจัดให้อยู่ในหมวดหมู่ใดหมวดหมู่หนึ่งเพียงหมวดหมู่เดียวเท่านั้น * *ตัวอย่าง:* การจำแนกเกรดของนักศึกษา (A, B, C, D, F), การจำแนกประเภทของสัตว์ (สุนัข, แมว, นก)

3. Multilabel Classification (การจำแนกประเภทแบบหลายป้ายกำกับ) เป็นปัญหาที่ข้อมูลหนึ่งตัวอย่างสามารถถูกจัดให้อยู่ในหลายหมวดหมู่ได้พร้อมๆ กัน * *ตัวอย่าง:* การจัดหมวดหมู่ภาพยนตร์ (ภาพยนตร์หนึ่งเรื่องอาจเป็นทั้ง Action, Sci-Fi และ Comedy พร้อมกัน)



แผนภาพจำลองขอบเขตการตัดสินใจ (Decision Boundary)

ตัวอย่างการประยุกต์ใช้ในบริบทประเทศไทย

เทคนิค Classification ถูกนำมาประยุกต์ใช้อย่างกว้างขวางในหลากหลายอุตสาหกรรมในประเทศไทย เพื่อช่วยในการตัดสินใจและลดภาระการทำงานของมนุษย์ พิจารณาตัวอย่างต่อไปนี้:

1. การตรวจจับอีเมลขยะ (Spam Detection) ในระบบเครือข่ายมหาวิทยาลัย ศูนย์คอมพิวเตอร์ของมหาวิทยาลัยขอนแก่น ต้องจัดการกับอีเมลที่ส่งถึงนักศึกษาและบุคลากรหลายแสนฉบับต่อวัน การใช้โมเดล Classification จะช่วยคัดกรองอีเมลหลอกลวง (Phishing) หรืออีเมลโฆษณา (Spam) ออกจากอีเมลสำคัญ * **Features (*X*):** ความถี่ของคำว่า “ด่วน”, “รหัสผ่าน”, “ถูกรางวัล”, โดเมนของผู้ส่ง, จำนวนลิงก์ในอีเมล * **Target (*Y*):** Spam (อีเมลขยะ) หรือ Ham (อีเมลปกติ)

2. การทำนายการยกเลิกบริการ (Customer Churn Prediction) ของเครือข่ายโทรคมนาคม ผู้ให้บริการเครือข่ายโทรศัพท์มือถือในประเทศไทย (เช่น AIS, True, Dtac) เผชิญกับการแข่งขันที่สูง การรักษาลูกค้าเดิมมีต้นทุนต่ำกว่าการหาลูกค้าใหม่ การทำนายว่าลูกค้าคนใดมีแนวโน้มจะย้ายค่าย (Churn) ทำให้บริษัทสามารถเสนอโปรโมชันพิเศษเพื่อดึงดูดลูกค้าไว้ได้ทันท่วงที (Knaflic, 2015) * **Features (*X*):** ระยะเวลาที่เป็นลูกค้า (เดือน), จำนวนนาทีที่โทรออก, จำนวนครั้งที่ติดต่อ Call Center, รูปแบบการชำระเงินล่าช้า * **Target (*Y*):** Churn (ยกเลิกบริการ) หรือ Retain (ใช้บริการต่อ)

3. การวินิจฉัยโรคทางการแพทย์ (Disease Diagnosis) โรงพยาบาลในประเทศไทยสามารถใช้ข้อมูลระบาดวิทยาและอาการของผู้ป่วย เพื่อคัดกรองความเสี่ยงของโรคไข้เลือดออก (Dengue Fever) ซึ่งเป็นโรคประจำถิ่นที่พบมากในช่วงฤดูฝน ช่วยให้แพทย์สามารถจัดลำดับความสำคัญของผู้ป่วยที่มีความเสี่ยงสูงได้ * **Features (*X*):** อุณหภูมิร่างกาย, จำนวนเกล็ดเลือด (Platelet count), อาการปวดกระบอกตา, การมีผื่นแดง * **Target (*Y*):** เป็นโรคไข้เลือดออก หรือ ไม่เป็นโรคไข้เลือดออก

4. การจำแนกสายพันธุ์พืชการเกษตร (Species Classification) ประเทศไทยเป็นประเทศเกษตรกรรมที่มีการส่งออกข้าวเป็นอันดับต้นๆ ของโลก การตรวจสอบความบริสุทธิ์ของสายพันธุ์ข้าวเป็นสิ่งสำคัญ โมเดล Classification สามารถใช้ภาพถ่ายเมล็ดข้าวเพื่อระบุสายพันธุ์ ลดความผิดพลาดจากการใช้สายตามนุษย์ตรวจวัด * **Features (*X*):** ความยาวของเมล็ดข้าว, ความกว้าง, อัตราส่วนความยาวต่อความกว้าง, ค่าสี (RGB) ของเมล็ด * **Target (*Y*):** ข้าวหอมมะลิ 105, ข้าวเหนียว กข6, ข้าวไรซ์เบอร์รี่ (ปัญหาแบบ Multiclass)

การสร้างโมเดล Classification เบื้องต้นด้วย Python

เพื่อให้เห็นภาพการทำงานของปัญหา Classification พิจารณาตัวอย่างการจำลองข้อมูลพฤติกรรมลูกค้า (Customer Churn) และการสร้างโมเดล Logistic Regression ซึ่งเป็นหนึ่งในอัลกอริทึมพื้นฐานสำหรับการทำ Binary Classification (VanderPlas, 2016)

```python
# นำเข้าไลบรารีที่จำเป็น
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# 1. การจำลองชุดข้อมูลพฤติกรรมลูกค้า (Customer Churn Dataset)
# สมมติว่ามีลูกค้า 1000 คน และมีตัวแปรอิสระ (Features) 4 ตัว
X, y = make_classification(
    n_samples=1000,
    n_features=4,
    n_informative=3, # ฟีเจอร์ที่มีผลต่อการตัดสินใจจริง
    n_redundant=1,
    random_state=42, # กำหนด seed เพื่อให้ผลลัพธ์เหมือนเดิมทุกครั้ง
    weights=[0.7, 0.3] # จำลองสถานการณ์ที่ลูกค้าส่วนใหญ่ไม่ยกเลิกบริการ (Class 0: 70%, Class 1: 30%)
)

# แปลงเป็น DataFrame เพื่อให้ง่ายต่อการทำความเข้าใจ
feature_names = ['ระยะเวลาใช้งาน(เดือน)', 'ค่าใช้จ่ายรายเดือน(บาท)', 'จำนวนครั้งที่แจ้งปัญหา', 'การชำระเงินล่าช้า']
df = pd.DataFrame(X, columns=feature_names)
df['ยกเลิกบริการ (Target)'] = y

print("ตัวอย่างข้อมูล 5 แถวแรก:")
print(df.head())
print("-" * 50)

# 2. แบ่งข้อมูลเป็นชุดฝึกสอน (Training Set) 80% และชุดทดสอบ (Testing Set) 20%
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. สร้างและฝึกสอนโมเดล Logistic Regression
model = LogisticRegression()
model.fit(X_train, y_train) # กระบวนการที่โมเดลเรียนรู้จากข้อมูล

# 4. นำโมเดลไปทำนายข้อมูลชุดทดสอบ
y_pred = model.predict(X_test)

# 5. ประเมินประสิทธิภาพของโมเดลเบื้องต้น
accuracy = accuracy_score(y_test, y_pred)
print(f"ความแม่นยำของโมเดล (Accuracy): {accuracy * 100:.2f}%")
```

```
print("รายงานการประเมินประสิทธิภาพ (Classification Report):")
# รายงานนี้จะแสดงค่า Precision, Recall และ F1-score สำหรับแต่ละ Class
print(classification_report(y_test, y_pred, target_names=['ไม่ยกเลิก (0)', 'ยกเลิก (1)']))
```

ผลลัพธ์:

```
ตัวอย่างข้อมูล 5 แถวแรก:
   ระยะเวลาใช้งาน(เดือน)  ค่าใช้จ่ายรายเดือน(บาท)  จำนวนครั้งที่แจ้งปัญหา  การชำระเงินล่าช้า  ยกเลิกบริการ (Target)
0        0.980832         0.408410                1.168098       -0.123571      1
1        0.445277        -1.986198                0.219744        1.869351      0
2       -1.892225        -1.032171                1.541761       -0.175732      0
3        0.081571        -2.216266               -0.718966        2.066282      0
4       -1.298140        -0.547365               -0.174167       -0.056450      0
--------------------------------------------------
ความแม่นยำของโมเดล (Accuracy): 90.50%

รายงานการประเมินประสิทธิภาพ (Classification Report):
               precision    recall  f1-score   support

ไม่ยกเลิก (0)       0.91      0.96      0.93       140
  ยกเลิก (1)       0.89      0.78      0.83        60

     accuracy                           0.91       200
    macro avg       0.90      0.87      0.88       200
 weighted avg       0.90      0.91      0.90       200
```

จากโค้ดด้านบน กระบวนการเริ่มต้นจากการเตรียมข้อมูล (Data Preparation) จากนั้นแบ่งข้อมูลออกเป็นชุดเรียนรู้และชุดทดสอบ เพื่อป้องกันการเกิดปัญหา Overfitting โมเดล `LogisticRegression` จะทำการเรียนรู้รูปแบบทางคณิตศาสตร์ที่แยกกลุ่มลูกค้าระหว่างกลุ่ม “ไม่ยกเลิก” และ “ยกเลิก” ออกจากกัน ก่อนที่จะนำโมเดลไปทดสอบกับข้อมูล `X_test` ที่โมเดลไม่เคยเห็นมาก่อน เพื่อประเมินความแม่นยำ (McKinney, 2022)

สรุปประเด็นสำคัญ (Key Takeaways)

- **Classification** เป็นปัญหาของ Supervised Learning ที่มุ่งเน้นการทำนายตัวแปรตาม (Target) ที่มีลักษณะเป็น **หมวดหมู่ (Categorical)** หรือค่าที่ไม่ต่อเนื่อง
- แตกต่างจาก Regression อย่างชัดเจน โดย Regression ใช้ทำนายค่าตัวเลขต่อเนื่อง (Continuous Numerical) ส่วน Classification ใช้ทำนายกลุ่มหรือคลาส (Class/Category)
- ประเภทของ Classification แบ่งหลักๆ ได้เป็น Binary (2 หมวดหมู่), Multiclass (มากกว่า 2 หมวดหมู่) และ Multilabel (หนึ่งข้อมูลอยู่ได้หลายหมวดหมู่)
- การประยุกต์ใช้ในโลกจริงมีหลากหลายครอบคลุมทุกอุตสาหกรรม เช่น การคัดกรองอีเมลขยะ การทำนายลูกค้ายกเลิกบริการ การวินิจฉัยโรค และการจำแนกสายพันธุ์ทางการเกษตร

คำถามท้าย Section

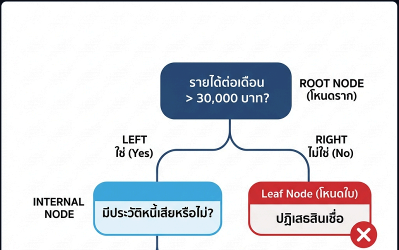
1. หากมหาวิทยาลัยต้องการสร้างระบบ AI เพื่อทำนายว่า “นักศึกษาแต่ละคนจะได้รับเกรดเฉลี่ยสะสม (GPAX) เมื่อสำเร็จการศึกษาเป็นเท่าใด (เช่น 3.45, 2.80)” ปัญหานี้จัดเป็นปัญหา Classification หรือไม่? จงอธิบายเหตุผลประกอบ
2. ในการสร้างโมเดลทำนายการเกิดโรคไข้เลือดออก (Binary Classification) หากโมเดลทำนายผิดพลาดระหว่าง “คนที่เป็นโรคแต่ทำนายว่าไม่เป็น” กับ “คนที่ไม่เป็นโรคแต่ทำนายว่าเป็น” ท่านคิดว่าความผิดพลาดรูปแบบใดส่งผลเสียต่อชีวิตผู้ป่วยมากกว่ากัน? และมีผลต่อการเลือกใช้เกณฑ์ประเมินประสิทธิภาพ (Metrics) อย่างไร?

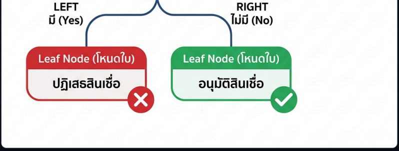
Decision Tree

Decision Tree หรือ ต้นไม้ตัดสินใจ เป็นอัลกอริทึมในกลุ่ม Supervised Learning ที่สามารถใช้ได้ทั้งกับงานจำแนกประเภท (Classification) และงานวิเคราะห์การถดถอย (Regression) ในบริบทของบทนี้ เราจะพิจารณาการใช้งาน Decision Tree สำหรับงานจำแนกประเภท (Decision Tree Classifier) เป็นหลัก

แนวคิดพื้นฐานของ Decision Tree คือการสร้างแบบจำลองการตัดสินใจในรูปแบบของโครงสร้างต้นไม้ (Tree structure) โดยอัลกอริทึมจะทำการแบ่งชุดข้อมูล (Partitioning) ออกเป็นกลุ่มย่อย ๆ ตามเงื่อนไขของตัวแปรคุณลักษณะ (Features) อย่างต่อเนื่อง จนกระทั่งข้อมูลในกลุ่มย่อยนั้นมีความเป็นเนื้อเดียวกัน (Homogeneous) มากที่สุด หรือจัดอยู่ในคลาส (Class) เดียวกัน (James et al., 2023)

พิจารณาตัวอย่างระบบการพิจารณาสินเชื่อของธนาคารพาณิชย์ในประเทศไทย โครงสร้างของ Decision Tree สามารถแสดงได้ดังแผนภาพด้านล่าง:



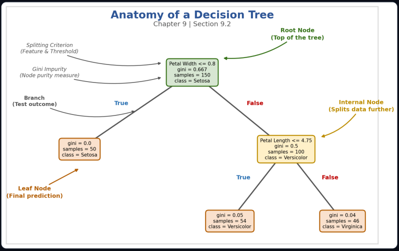


Decision Tree Structure

จากแผนภาพ โหนดบนสุดเรียกว่า **Root Node** ซึ่งเป็นจุดเริ่มต้นของการตัดสินใจ โหนดที่อยู่ระหว่างทางเรียกว่า **Internal Node** ซึ่งทำหน้าที่เป็นเงื่อนไขในการแบ่งข้อมูล และโหนดปลายทางเรียกว่า **Leaf Node** ซึ่งเป็นผลลัพธ์ของการจำแนกประเภท (Class label)

เกณฑ์การแบ่งข้อมูล (Splitting Criteria)

คำถามสำคัญในการสร้าง Decision Tree คือ อัลกอริทึมจะทราบได้อย่างไรว่าควรเลือกตัวแปรใดและค่าใดมาเป็นเงื่อนไขในการแบ่งข้อมูลที่แต่ละโหนด? คำตอบคือ อัลกอริทึมจะประเมินจาก “ความบริสุทธิ์” (Purity) ของข้อมูลหลังจากการแบ่ง โดยอาศัยมาตรวัดทางคณิตศาสตร์ 2 รูปแบบหลัก ได้แก่ Gini Impurity และ Information Gain (Entropy) (VanderPlas, 2016)



Anatomy of a Decision Tree

Gini Impurity

Gini Impurity เป็นมาตรวัดความไม่บริสุทธิ์ของข้อมูลในโหนด หากโหนดมีข้อมูลที่มาจากคลาสเดียวกันทั้งหมด ค่า Gini Impurity จะเท่ากับ 0 (บริสุทธิ์ที่สุด) สมการของ Gini Impurity คำนวณได้ดังนี้:

$$Gini = 1 - \sum_{i=1}^{C} (p_i)^2$$

โดยที่: - C คือ จำนวนคลาสทั้งหมด - p_i คือ สัดส่วนของข้อมูลที่อยู่ในคลาส i ภายในโหนดนั้น

ตัวอย่างการคำนวณ: สมมติว่ามีข้อมูลลูกค้าสมัครบัตรเครดิต 10 คน ในโหนดหนึ่ง ประกอบด้วยลูกค้าที่ “อนุมัติ” 6 คน และ “ไม่อนุมัติ” 4 คน - $p_{อนุมัติ}$ = 6/10 = 0.6 - $p_{ไม่อนุมัติ}$ = 4/10 = 0.4 - $Gini$ = 1 − (0.6^2 + 0.4^2) = 1 − (0.36 + 0.16) = 1 − 0.52 = 0.48

Information Gain และ Entropy

อีกหนึ่งวิธีในการวัดคุณภาพของการแบ่งข้อมูลคือการใช้ Entropy ซึ่งมีรากฐานมาจากทฤษฎีข้อมูล (Information Theory) Entropy คือมาตรวัดความไม่แน่นอนหรือความไร้ระเบียบของข้อมูล:

$$Entropy = -\sum_{i=1}^{C} p_i \log_2(p_i)$$

จากตัวอย่างลูกค้า 10 คนด้านบน: - $Entropy$ = −(0.6log$_2$(0.6) + 0.4log$_2$(0.4)) ≈ 0.971

เมื่อเราทราบค่าความไม่บริสุทธิ์ของโหนดเริ่มต้นแล้ว อัลกอริทึมจะคำนวณ **Information Gain (IG)** ซึ่งคือผลต่างระหว่างความไม่บริสุทธิ์ก่อนการแบ่ง (Parent Node) และค่าเฉลี่ยถ่วงน้ำหนักของความไม่บริสุทธิ์หลังการแบ่ง (Child Nodes) อัลกอริทึมจะเลือกตัวแปรและจุดแบ่งที่ให้ค่า Information Gain สูงที่สุด (หรือลด Impurity ได้มากที่สุด) ในแต่ละรอบของการสร้างโหนด

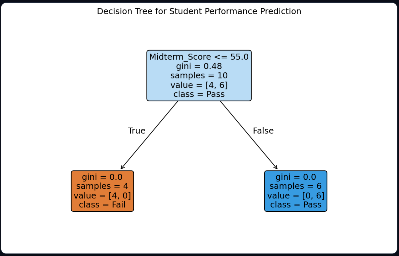
การสร้างแบบจำลองและการอ่านผล (Visualization)

เพื่อทำความเข้าใจการทำงานของ Decision Tree ในทางปฏิบัติ เราจะใช้ไลบรารี `scikit-learn` ในการสร้างแบบจำลอง โดยพิจารณาชุดข้อมูลจำลองของนักศึกษาสาขาวิทยาการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น เพื่อทำนายว่านักศึกษาจะ “ผ่าน” (Pass = 1) หรือ “ตก” (Fail = 0) ในรายวิชา Introduction to Data Science โดยพิจารณาจากคะแนนสอบกลางภาค (Midterm_Score) และจำนวนครั้งที่ขาดเรียน (Absences)

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree

# 1. สร้างชุดข้อมูลจำลอง (นักศึกษา 10 คน)
data = {
    'Midterm_Score': [85, 45, 60, 90, 35, 75, 50, 80, 40, 65],
    'Absences': [0, 5, 2, 1, 6, 1, 4, 0, 7, 3],
    'Pass': [1, 0, 1, 1, 0, 1, 0, 1, 0, 1] # 1: ผ่าน, 0: ตก
}
df = pd.DataFrame(data)

# กำหนด Features (X) และ Target (y)
X = df[['Midterm_Score', 'Absences']]
y = df['Pass']

# 2. สร้างและฝึกสอนแบบจำลอง Decision Tree
# กำหนด criterion='gini' และกำหนด random_state เพื่อให้ผลลัพธ์คงที่
clf = DecisionTreeClassifier(criterion='gini', random_state=42)
clf.fit(X, y)
```

```python
# 3. วาดแผนภาพ Decision Tree (Visualization)
plt.figure(figsize=(10, 6))
plot_tree(clf,
          feature_names=['Midterm_Score', 'Absences'],
          class_names=['Fail', 'Pass'],
          filled=True,
          rounded=True,
          fontsize=12)
plt.title("Decision Tree for Student Performance Prediction")
plt.show()
```

ผลลัพธ์:



Decision Tree Visualization
จากการรันโค้ดด้านบน ฟังก์ชัน `plot_tree` จะแสดงแผนภาพต้นไม้ ซึ่งแต่ละโหนดจะมีข้อมูลที่สำคัญดังนี้: 1. **เงื่อนไข (เช่น `Midterm_Score ≤ 55.0`):** กฎที่ใช้แบ่งข้อมูล หากเป็นจริง (True) ข้อมูลจะไปทางซ้าย หากเป็นเท็จ (False) ข้อมูลจะไปทางขวา 2. `gini`**:** ค่า Gini Impurity ของโหนดนั้น 3. `samples`**:** จำนวนตัวอย่างข้อมูลที่อยู่ในโหนดนั้น 4. `value`**:** จำนวนตัวอย่างที่ถูกแยกตามคลาส เช่น `[4, 6]` หมายถึง มีนักศึกษาที่ตก 4 คน และผ่าน 6 คน 5. `class`**:** คลาสที่มีจำนวนมากที่สุดในโหนดนั้น (Majority class)

วิธีการอ่าน Tree: จาก Root Node หากนักศึกษามีคะแนนกลางภาคน้อยกว่าหรือเท่ากับ 55 คะแนน (ไปทางซ้าย) จะพบว่าเป็น Leaf Node ที่มี `value = [4, 0]` และ `class = Fail` หมายความว่า นักศึกษาทั้ง 4 คนในกลุ่มนี้สอบตกทั้งหมด (Gini = 0.0 บริสุทธิ์สมบูรณ์) ในทางกลับกัน หากคะแนนมากกว่า 55 (ไปทางขวา) จะพบนักศึกษา 6 คนที่สอบผ่านทั้งหมด `value = [0, 6]` และ `class = Pass` (Gini = 0.0)

ไฮเปอร์พารามิเตอร์ (Hyperparameters) และปัญหา Overfitting

ข้อควรระวังที่สำคัญที่สุดในการใช้ Decision Tree คือปัญหา **Overfitting** (การเรียนรู้จำเพาะเจาะจงเกินไป) หากเราปล่อยให้อัลกอริทึมสร้างต้นไม้โดยไม่มีการควบคุม ต้นไม้จะแตกกิ่งก้านสาขาไปเรื่อย ๆ จนกระทั่งทุก Leaf node มี Gini Impurity เป็น 0 ซึ่งหมายความว่าแบบจำลองจดจำชุดข้อมูลฝึกสอน (Training data) ได้สมบูรณ์แบบ แต่จะสูญเสียความสามารถในการสรุปความ (Generalization) เมื่อนำไปใช้กับข้อมูลใหม่ (Unseen data) (McKinney, 2022)

เพื่อป้องกันปัญหา Overfitting เราจำเป็นต้องปรับแต่งค่าไฮเปอร์พารามิเตอร์ (Hyperparameters) เพื่อจำกัดการเติบโตของต้นไม้ (Tree Pruning) พารามิเตอร์ที่สำคัญใน `scikit-learn` ได้แก่:

ไฮเปอร์พารามิเตอร์	คำอธิบาย	ผลกระทบเมื่อปรับค่า
`max_depth`	ความลึกสูงสุดของต้นไม้ (จำนวนชั้นจาก Root ถึง Leaf)	ถ้าน้อยเกินไปจะเกิด Underfitting ถ้ามากเกินไปจะเกิด Overfitting
`min_samples_split`	จำนวนตัวอย่างขั้นต่ำที่โหนดต้องมี เพื่อที่จะสามารถแตกกิ่งต่อไปได้	ค่าที่สูงขึ้นช่วยป้องกันไม่ให้โมเดลสร้างกฎเฉพาะเจาะจงจากข้อมูลจำนวนน้อย
`min_samples_leaf`	จำนวนตัวอย่างขั้นต่ำที่โหนดใบ (Leaf node) ต้องมี	ช่วยลดผลกระทบจากข้อมูลที่เป็น Outlier หรือ Noise

ตัวอย่างการตั้งค่า Hyperparameters ใน Python:

```python
# การสร้าง Decision Tree โดยจำกัดความลึกไม่เกิน 3 ชั้น
# และโหนดใบต้องมีข้อมูลอย่างน้อย 5 ตัวอย่าง
pruned_clf = DecisionTreeClassifier(
    criterion='entropy',       # ใช้ Information Gain
    max_depth=3,               # จำกัดความลึก
    min_samples_split=10,      # ต้องมี 10 samples ถึงจะแบ่งโหนดได้
    min_samples_leaf=5,        # โหนดใบต้องมีอย่างน้อย 5 samples
    random_state=42
)
# pruned_clf.fit(X_train, y_train)
```

> **หมายเหตุ:** โค้ดด้านบนแสดงเฉพาะการตั้งค่า Hyperparameters เท่านั้น ไม่ได้ทำการ `fit()` จริง เพื่อให้นักศึกษานำไปประยุกต์กับชุดข้อมูลจริงในการฝึกหัด

ข้อดีและข้อเสียของ Decision Tree

การเลือกใช้ Decision Tree ควรพิจารณาจากบริบทของงานและลักษณะของข้อมูล โดยมีข้อดีและข้อเสียดังนี้ (Grus, 2019):

ข้อดี (Pros): 1. **ความสามารถในการอธิบาย (Interpretability):** เป็นแบบจำลองแบบ White-box ที่มนุษย์สามารถเข้าใจเหตุผลเบื้องหลังการตัดสินใจได้ง่ายผ่านแผนภาพ (Visualization) ซึ่งสำคัญมากในวงการแพทย์หรือการเงิน 2. **ต้องการการเตรียมข้อมูลน้อย (Minimal Data Preparation):** ไม่จำเป็นต้องทำ Data Scaling (Standardization/Normalization) เนื่องจากอัลกอริทึมพิจารณาการแบ่งข้อมูลตามลำดับค่า (Rank) ไม่ใช่ระยะห่างทางคณิตศาสตร์ 3. **รองรับความสัมพันธ์แบบไม่เป็นเชิงเส้น (Non-linear relationships):** สามารถจัดการกับชุดข้อมูลที่ตัวแปรมีความสัมพันธ์ที่ซับซ้อนและไม่เป็นเส้นตรงได้ดี

ข้อเสีย (Cons): 1. **ความไม่เสถียร (Instability / High Variance):** การเปลี่ยนแปลงชุดข้อมูลฝึกสอนเพียงเล็กน้อย อาจทำให้โครงสร้างของต้นไม้เปลี่ยนไปอย่างสิ้นเชิง 2. **เสี่ยงต่อการเกิด Overfitting สูง:** หากไม่มีการตั้งค่า ไฮเปอร์พารามิเตอร์ (Hyperparameters) อย่างเหมาะสม 3. **ขอบเขตการตัดสินใจเป็นมุมฉาก (Orthogonal Decision Boundaries):** เนื่องจาก Decision Tree แบ่งข้อมูลทีละตัวแปรตามแกน (Axis-aligned splits) จึงอาจไม่มีประสิทธิภาพหากขอบเขตการจำแนกประเภทที่แท้จริงเป็นเส้นทแยงมุมหรือเส้นโค้ง

สรุปประเด็นสำคัญ

- Decision Tree เป็นอัลกอริทึมที่จำแนกประเภทข้อมูลโดยการสร้างกฎการตัดสินใจแบบลำดับชั้น (Tree structure)
- อัลกอริทึมเลือกจุดแบ่งข้อมูลโดยพิจารณาจากการลดค่าความไม่บริสุทธิ์ (Impurity) ซึ่งวัดจาก Gini Impurity หรือ Information Gain (Entropy)
- การอ่านแผนภาพ Decision Tree ช่วยให้เข้าใจตรรกะของแบบจำลองได้อย่างชัดเจน (Interpretability)
- ปัญหาหลักของ Decision Tree คือ Overfitting ซึ่งสามารถแก้ไขได้โดยการปรับตั้งค่า Hyperparameters เช่น `max_depth`, `min_samples_split` และ `min_samples_leaf`

คำถามท้าย Section

1. หากโหนดหนึ่งใน Decision Tree มีตัวอย่างข้อมูลที่มาจากคลาส A จำนวน 50 ตัวอย่าง และคลาส B จำนวน 50 ตัวอย่าง ค่า Gini Impurity ของโหนดนี้จะมีค่าเท่าใด? และค่านี้บ่งบอกถึงอะไร?
2. สมมติว่านักศึกษาใช้ Decision Tree ทำนายการซื้อสินค้าบน e-commerce แล้วพบว่าโมเดลมีความแม่นยำบน Training data 99% แต่มีความแม่นยำบน Test data เพียง 60% ปรากฏการณ์นี้เรียกว่าอะไร และนักศึกษาจะปรับค่า `max_depth` อย่างไรเพื่อแก้ไขปัญหานี้?
3. จงอธิบายเหตุผลว่าทำไม Decision Tree จึงไม่จำเป็นต้องทำ Data Normalization ก่อนการสร้างแบบจำลอง เมื่อเทียบกับอัลกอริทึมอย่าง Logistic Regression หรือ K-Nearest Neighbors (KNN)?

k-Nearest Neighbors

k-Nearest Neighbors (k-NN) เป็นหนึ่งในอัลกอริทึมพื้นฐานที่สำคัญและเรียบง่ายที่สุดในกลุ่ม Supervised Learning ซึ่งสามารถนำมาใช้ได้ทั้งกับปัญหาการจำแนกประเภท (Classification) และการวิเคราะห์การถดถอย (Regression) แนวคิดหลักของ k-NN ตั้งอยู่บนสมมติฐานที่ว่าข้อมูลที่มีลักษณะคล้ายคลึงกันมักจะอยู่ในตำแหน่งที่ใกล้เคียงกันในปริภูมิของลักษณะเฉพาะ (Feature Space) (James et al., 2023)

อัลกอริทึม k-NN จัดอยู่ในกลุ่ม **Instance-based Learning** หรือ **Lazy Learning** เนื่องจากไม่มีการสร้างโมเดลทางคณิตศาสตร์ที่ชัดเจนในขั้นตอนการฝึกสอน (Training Phase) แต่โมเดลจะทำการจดจำข้อมูลชุดฝึกสอนทั้งหมดไว้ และจะทำการคำนวณเพื่อจำแนกประเภทก็ต่อเมื่อมีข้อมูลใหม่ (Test Instance) เข้ามาในระบบเท่านั้น

กลไกการทำงานของอัลกอริทึม (Algorithm: Distance and Voting)

การทำงานของ k-NN สำหรับปัญหาการจำแนกประเภท ประกอบด้วย 3 ขั้นตอนหลัก ดังนี้:

1. **การคำนวณระยะห่าง (Distance Calculation):** คำนวณระยะห่างระหว่างข้อมูลใหม่ที่ต้องการทำนาย กับข้อมูลชุดฝึกสอนทุกจุดในระบบ
2. **การค้นหาเพื่อนบ้าน (Finding Nearest Neighbors):** เลือกข้อมูลชุดฝึกสอนที่มีระยะห่างน้อยที่สุดจำนวน k ตัว (โดยที่ k คือพารามิเตอร์ที่กำหนดโดยผู้ใช้งาน)
3. **การลงคะแนนเสียง (Majority Voting):** ตรวจสอบว่าเพื่อนบ้านทั้ง k ตัวนั้นอยู่ในคลาส (Class) ใดมากที่สุด ข้อมูลใหม่จะถูกจัดให้อยู่ในคลาสที่ได้รับคะแนนเสียงข้างมาก

การวัดระยะห่าง (Distance Metrics) การคำนวณระยะห่างเป็นหัวใจสำคัญของ k-NN ฟังก์ชันการวัดระยะห่างที่นิยมใช้กันอย่างแพร่หลายได้แก่:

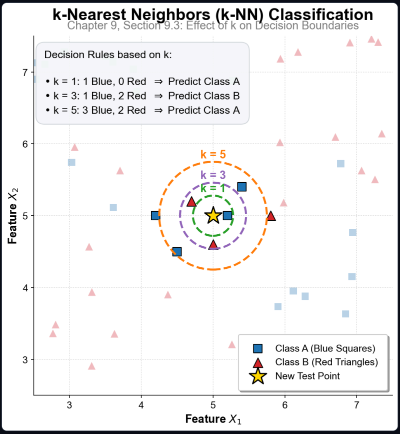
- **Euclidean Distance:** เป็นการวัดระยะทางเป็นเส้นตรงระหว่างจุดสองจุด นิยมใช้กับข้อมูลเชิงปริมาณ (Continuous Variables)

$$d(p,q) = \sqrt{\sum_{i=1}^{n}(p_i - q_i)^2}$$

- **Manhattan Distance:** เป็นการวัดระยะทางตามแนวแกน (คล้ายกับการเดินตามบล็อกถนน) เหมาะสำหรับข้อมูลที่มีลักษณะเป็น Grid หรือข้อมูลที่มี Outliers สูง

$$d(p,q) = \sum_{i=1}^{n}|p_i - q_i|$$

แผนภาพที่ 9.3.1: แสดงแนวคิดการลงคะแนนเสียงของ k-NN ในปริภูมิ 2 มิติ



KNN Visualization

พิจารณาจากแผนภาพ: หากกำหนด k = 3 และพบว่าเพื่อนบ้านที่ใกล้ที่สุด 3 ตัวประกอบด้วย Class A จำนวน 1 ตัว และ Class B จำนวน 2 ตัว ระบบจะทำการลงคะแนนเสียง (Voting) และสรุปผลว่าข้อมูลใหม่ `[?]` ควรถูกจัดอยู่ใน **Class B**

ความสำคัญของการปรับมาตราส่วนข้อมูล (Feature Scaling: StandardScaler)

ข้อควรระวังที่สำคัญที่สุดประการหนึ่งในการใช้งาน k-NN คือ อัลกอริทึมนี้มีความอ่อนไหวต่อขนาด (Scale) ของข้อมูลเป็นอย่างมาก (VanderPlas, 2016) เนื่องจากพึ่งพาการคำนวณระยะห่างทางคณิตศาสตร์โดยตรง

พิจารณาตัวอย่างระบบพิจารณาอนุมัติสินเชื่อของธนาคารพาณิชย์ในประเทศไทย ซึ่งมี Features 2 ตัว ได้แก่: 1. **อายุ (Age):** มีค่าอยู่ในช่วง 20 - 60 ปี 2. **รายได้ต่อเดือน (Monthly Income):** มีค่าอยู่ในช่วง 15,000 - 200,000 บาท

หากคำนวณ Euclidean Distance โดยไม่ปรับมาตราส่วน ความแตกต่างของ “รายได้” ซึ่งมีหลักแสน จะครอบงำ (Dominate) ความแตกต่างของ “อายุ” ซึ่งมีหลักสิบอย่างสมบูรณ์ ทำให้โมเดลเพิกเฉยต่อความสำคัญของอายุไปโดยปริยาย

ตารางที่ 9.3.1: ผลกระทบของ Scale ต่อการคำนวณระยะห่าง

ลูกค้า	อายุ (ปี)	รายได้ (บาท)	ระยะห่างจากลูกค้า A (ไม่ปรับ Scale)
A (เป้าหมาย)	25	30,000	0
B	26	35,000	$\sqrt{(25-26)^2 + (30000-35000)^2} \approx 5000$
C	50	30,000	$\sqrt{(25-50)^2 + (30000-30000)^2} = 25$

จากตาราง ลูกค้า A และ C มีอายุห่างกันถึง 25 ปี แต่ระยะห่างทางคณิตศาสตร์กลับน้อยกว่าลูกค้า A และ B ที่อายุห่างกันเพียง 1 ปี แต่รายได้ต่างกัน 5,000 บาท

เพื่อแก้ปัญหานี้ เราต้องทำ **Standardization** (หรือ Z-score normalization) ผ่าน `StandardScaler` ซึ่งจะปรับให้ทุก Feature มีค่าเฉลี่ย (Mean) เท่ากับ 0 และส่วนเบี่ยงเบนมาตรฐาน (Standard Deviation) เท่ากับ 1

$$z = \frac{x - \mu}{\sigma}$$

การเลือกค่า k ที่เหมาะสม (Choosing k: The Elbow Method)

พารามิเตอร์ k (จำนวนเพื่อนบ้าน) ควบคุมความซับซ้อนของโมเดล (Model Complexity) และส่งผลโดยตรงต่อปัญหา Bias-Variance Tradeoff (McKinney, 2022) * **หาก k มีค่าน้อยเกินไป (เช่น k = 1):** โมเดลจะมีความซับซ้อนสูง สนใจรายละเอียดของทุกจุดข้อมูล ทำให้เกิดปัญหา **Overfitting** (Low Bias, High Variance) ขอบเขตการตัดสินใจ (Decision Boundary) จะขรุขระและอ่อนไหวต่อสัญญาณรบกวน (Noise) * **หาก k มีค่ามากเกินไป:** โมเดลจะเรียบง่ายเกินไป เกิดปัญหา **Underfitting** (High Bias, Low Variance) ขอบเขตการตัดสินใจจะราบเรียบ และอาจจัดประเภทข้อมูลกลุ่มเล็กผิดพลาด

วิธีการที่เป็นมาตรฐานในการค้นหาค่า k ที่เหมาะสมคือการใช้ **Elbow Method** ร่วมกับการประเมินค่าความคลาดเคลื่อน (Error Rate) วิธีการคือทดลองรันโมเดลด้วยค่า k ที่แตกต่างกัน (เช่น ตั้งแต่ 1 ถึง 20) คำนวณ Error Rate บนชุดข้อมูลทดสอบ (Test Set) และพล็อตลงบนกราฟ

จุดที่เหมาะสมที่สุด (Optimal k) คือจุดที่กราฟเริ่มหักมุมคล้าย “ข้อศอก” (Elbow) ซึ่งเป็นจุดที่ Error Rate ลดลงจนถึงระดับที่เสถียร การเพิ่มค่า k หลังจากจุดนี้จะไม่ช่วยลดความคลาดเคลื่อนอย่างมีนัยสำคัญ *ข้อแนะนำเพิ่มเติม:* ในกรณีที่จำแนกข้อมูล 2 คลาส ควรเลือกค่า k เป็นเลขคี่ เพื่อป้องกันปัญหาคะแนนเสียงเท่ากัน (Tie)

การถ่วงน้ำหนักด้วยระยะทาง (Distance Weighting)

ในอัลกอริทึม k-NN แบบดั้งเดิม (Uniform Voting) เพื่อนบ้านทั้ง k ตัวจะมีสิทธิ์ออกเสียงเท่าเทียมกัน (1 เสียงต่อ 1 จุดข้อมูล) ไม่ว่าจุดข้อมูลนั้นจะอยู่ใกล้หรือไกลจากข้อมูลเป้าหมายก็ตาม ซึ่งในบางบริบทอาจไม่สมเหตุสมผล

Distance Weighting เป็นการปรับปรุงอัลกอริทึมโดยให้คะแนนเสียงของเพื่อนบ้านแปรผกผันกับระยะห่าง (Inverse Distance Weighting) หมายความว่า เพื่อนบ้านที่อยู่ใกล้กว่า จะมีอิทธิพลต่อการตัดสินใจมากกว่าเพื่อนบ้านที่อยู่ไกลออกไป (Grus, 2019)

สมการน้ำหนัก (Weight) พื้นฐาน:

$$w_i = \frac{1}{d(x, x_i)}$$

โดยที่ $d(x, x_i)$ คือระยะห่างระหว่างข้อมูลใหม่ x และเพื่อนบ้าน x_i

การใช้ Distance Weighting มีประโยชน์อย่างยิ่งเมื่อข้อมูลมีการกระจายตัวไม่สม่ำเสมอ หรือเมื่อเราจำเป็นต้องใช้ค่า k ที่สูงขึ้นเพื่อลดผลกระทบจาก Noise

ข้อดีและข้อเสียของ k-NN (Pros and Cons)

ข้อดี (Pros): 1. **เข้าใจง่ายและนำไปใช้งานได้ทันที:** อัลกอริทึมไม่มีความซับซ้อนทางคณิตศาสตร์ และไม่ต้องใช้เวลาในการฝึกสอนโมเดล (No Training Time) 2. **รองรับ Multi-class Classification โดยธรรมชาติ:** ไม่ต้องปรับแต่งโครงสร้างเพื่อรองรับคลาสที่มากกว่า 2 คลาส 3. **ปรับตัวตามข้อมูลใหม่ได้ทันที:** เนื่องจากเป็น Lazy Learning เมื่อมีการเพิ่มข้อมูลใหม่ลงในฐานข้อมูล โมเดลสามารถนำมาใช้พิจารณาได้ทันทีโดยไม่ต้อง Retrain

ข้อเสีย (Cons): 1. **ใช้ทรัพยากรการประมวลผลสูงในขั้นตอนการทำนาย (High Inference Cost):** ต้องคำนวณระยะห่างระหว่างข้อมูลใหม่กับข้อมูลชุดฝึกสอน *ทุกตัว* ซึ่งไม่มีประสิทธิภาพหากชุดข้อมูลมีขนาดใหญ่ (Big Data) 2. **ใช้หน่วยความจำสูง (High Memory Usage):** ต้องเก็บข้อมูลชุดฝึกสอนทั้งหมดไว้ในหน่วยความจำ (RAM) 3. **เผชิญกับปัญหา “คำสาปของมิติ” (Curse of Dimensionality):** เมื่อจำนวน Features เพิ่มขึ้น ปริภูมิข้อมูลจะขยายใหญ่ขึ้นอย่างมหาศาล ทำให้ระยะห่างระหว่างจุดข้อมูลทุกจุดมีค่าใกล้เคียงกัน ส่งผลให้ประสิทธิภาพของ k-NN ลดลงอย่างรวดเร็ว

การนำไปใช้งานด้วย Python (Implementation)

พิจารณาตัวอย่างการประยุกต์ใช้ k-NN ในบริบทของมหาวิทยาลัยขอนแก่น เพื่อทำนายว่านักศึกษาชั้นปีที่ 1 สาขาวิทยาการคอมพิวเตอร์ จะสามารถสอบผ่านรายวิชา Programming ได้หรือไม่ โดยพิจารณาจาก “จำนวนชั่วโมงที่อ่านหนังสือต่อสัปดาห์” และ “จำนวนชั่วโมงที่เล่นเกมต่อสัปดาห์”

```python
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

# 1. จำลองชุดข้อมูล (Synthetic Data)
np.random.seed(42)
# สร้างข้อมูลนักศึกษา 200 คน
# Features: [ชั่วโมงอ่านหนังสือ, ชั่วโมงเล่นเกม]
study_hours = np.random.normal(15, 5, 200)
game_hours = np.random.normal(20, 8, 200)

# เงื่อนไขสมมติ: ถ้าอ่านหนังสือ > เกม/1.5 มักจะสอบผ่าน (1 = ผ่าน, 0 = ตก)
labels = (study_hours > (game_hours / 1.5)).astype(int)

# เพิ่ม Noise ให้ข้อมูลมีความสมจริง
noise = np.random.randint(0, 2, 20)
labels[:20] = noise

df = pd.DataFrame({
    'Study_Hours': study_hours,
```

```python
    'Game_Hours': game_hours,
    'Pass': labels
})

# 2. แบ่งข้อมูลเป็นชุดฝึกสอนและชุดทดสอบ (80/20)
X = df[['Study_Hours', 'Game_Hours']]
y = df['Pass']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. การปรับมาตราส่วนข้อมูล (Feature Scaling) - ขั้นตอนที่ขาดไม่ได้สำหรับ k-NN
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # Fit และ Transform ชุดฝึกสอน
X_test_scaled = scaler.transform(X_test)       # Transform ชุดทดสอบด้วยค่า Mean/SD จากชุดฝึกสอน

# 4. การค้นหาค่า k ด้วย Elbow Method
error_rates = []
k_range = range(1, 20, 2) # ทดสอบเฉพาะเลขคี่เพื่อป้องกันการ Tie

for k in k_range:
    # กำหนด weight='distance' เพื่อถ่วงน้ำหนักตามระยะห่าง
    knn = KNeighborsClassifier(n_neighbors=k, weights='distance')
    knn.fit(X_train_scaled, y_train)
    y_pred_k = knn.predict(X_test_scaled)
    error = 1 - accuracy_score(y_test, y_pred_k)
    error_rates.append(error)

# หาค่า k ที่ให้ Error ต่ำที่สุด (จำลองการดู Elbow)
optimal_k = k_range[np.argmin(error_rates)]
print(f"ค่า k ที่เหมาะสมที่สุดจากการทดสอบคือ: {optimal_k}")

# 5. สร้างและทดสอบโมเดลสุดท้ายด้วย Optimal k
final_knn = KNeighborsClassifier(n_neighbors=optimal_k, weights='distance')
final_knn.fit(X_train_scaled, y_train)
y_pred_final = final_knn.predict(X_test_scaled)

# 6. ประเมินผลลัพธ์
print("--- รายงานผลการจำแนกประเภท (Classification Report) ---")
print(classification_report(y_test, y_pred_final, target_names=['Fail (0)', 'Pass (1)']))
```

ผลลัพธ์:

```
ค่า k ที่เหมาะสมที่สุดจากการทดสอบคือ: 1

--- รายงานผลการจำแนกประเภท (Classification Report) ---
              precision    recall  f1-score   support

    Fail (0)       0.94      0.88      0.91        17
    Pass (1)       0.92      0.96      0.94        23

    accuracy                           0.93        40
   macro avg       0.93      0.92      0.92        40
weighted avg       0.93      0.93      0.92        40
```

จากการรันโค้ดด้านบน เราได้ปรับสเกลข้อมูล ป้องกันผลกระทบจากหน่วยวัดที่ต่างกัน และใช้ลูปเพื่อหาค่า *k* ที่เหมาะสมที่สุด พร้อมทั้งกำหนดให้เพื่อนบ้านที่อยู่ใกล้มีสิทธิออกเสียงมากกว่า (`weights='distance'`) ซึ่งเป็นแนวปฏิบัติที่ดีที่สุด (Best Practices) ในการใช้งาน k-NN

สรุปประเด็นสำคัญ (Key Takeaways)

- **Instance-based Learning:** k-NN ไม่มีการสร้างสมการคณิตศาสตร์ในขั้นตอนการฝึกสอน แต่ใช้การจดจำข้อมูลและคำนวณระยะห่างเมื่อต้องการทำนาย
- **Scale Sensitivity:** อัลกอริทึมอ่อนไหวต่อขนาดของข้อมูลอย่างมาก การใช้ `StandardScaler` เพื่อปรับข้อมูลให้อยู่ในสเกลเดียวกันเป็นขั้นตอนที่บังคับทำก่อนใช้ k-NN
- **Hyperparameter Tuning:** ค่า *k* ที่น้อยไปทำให้เกิด Overfitting ค่า *k* ที่มากไปทำให้เกิด Underfitting การใช้ Elbow Method ช่วยค้นหาจุดสมดุลที่เหมาะสมได้
- **Distance Weighting:** การปรับให้สิทธิการลงคะแนนของเพื่อนบ้านแปรผกผันกับระยะทาง ช่วยเพิ่มความแม่นยำในกรณีที่ข้อมูลมีการกระจายตัวไม่สม่ำเสมอ
- **Curse of Dimensionality:** k-NN เหมาะกับชุดข้อมูลที่มีจำนวน Features ไม่มากนัก หากมี Features มากเกินไป ประสิทธิภาพและระยะเวลาในการคำนวณจะแย่ลงอย่างมีนัยสำคัญ

คำถามท้าย Section

1. หากท่านต้องการนำ k-NN ไปใช้พัฒนาระบบคัดกรองผู้ป่วยเบาหวานจากฐานข้อมูลสาธารณสุข ซึ่งประกอบด้วย Features: อายุ (ปี), ระดับน้ำตาลในเลือด (mg/dL), และดัชนีมวลกาย (BMI) เหตุใดการข้ามขั้นตอน Feature Scaling จึงอาจทำให้ระบบคัดกรองผู้ป่วยผิดพลาดได้? จงอธิบายเชิงคณิตศาสตร์
2. จงอธิบายความสัมพันธ์ระหว่างค่า *k* ใน k-NN กับปัญหา Bias-Variance Tradeoff และหากชุดข้อมูลของท่านมี Outliers ปะปนอยู่จำนวนมาก ท่านควรเลือกใช้ค่า *k* ที่มีค่าน้อยหรือค่ามาก? เพราะเหตุใด?
3. พิจารณาข้อจำกัดของ k-NN ในด้าน “High Inference Cost” ในฐานะนักวิทยาการคอมพิวเตอร์ ท่านมีแนวคิดในการปรับปรุงโครงสร้างข้อมูล (Data Structure) อย่างไรเพื่อให้การค้นหาเพื่อนบ้านที่ใกล้ที่สุด *k* ตัว ทำได้รวดเร็วกว่าการคำนวณระยะห่างกับข้อมูลทุกจุดแบบตรงไปตรงมา (Brute-force)?

Classification Metrics

ในการพัฒนาโมเดลการเรียนรู้ของเครื่อง (Machine Learning) สำหรับปัญหาการจำแนกประเภท (Classification) การวัดประสิทธิภาพของโมเดลเป็นขั้นตอนที่มีความสำคัญอย่างยิ่งยวด การทราบเพียงว่าโมเดลทำนายถูกหรือผิดนั้นไม่เพียงพอต่อการนำไปใช้งานจริง (Production) เนื่องจากความผิดพลาดแต่ละประเภทอาจส่งผลกระทบและมีต้นทุน (Cost of Error) ที่แตกต่างกันอย่างสิ้นเชิง การเลือกใช้ตัวชี้วัด (Metrics) ที่เหมาะสมจึงเป็นทักษะสำคัญของนักวิทยาศาสตร์ข้อมูล (Grus, 2019)

เมทริกซ์ความสับสน (Confusion Matrix)

รากฐานของการวัดประสิทธิภาพในปัญหาการจำแนกประเภทเริ่มต้นที่ **เมทริกซ์ความสับสน (Confusion Matrix)** ซึ่งเป็นตารางที่แสดงผลการเปรียบเทียบระหว่าง “ค่าจริง (Actual Values)” และ “ค่าที่โมเดลทำนายได้ (Predicted Values)” การวิเคราะห์ตารางนี้จะช่วยให้เห็นรายละเอียดว่าโมเดลมีความสับสนในการจำแนกคลาสใดไปเป็นคลาสใดบ้าง (James et al., 2023)

สำหรับปัญหาการจำแนกประเภทแบบสองคลาส (Binary Classification) ซึ่งประกอบด้วยคลาสบวก (Positive Class) และคลาสลบ (Negative Class) Confusion Matrix จะมีขนาด 2x2 ดังตารางต่อไปนี้

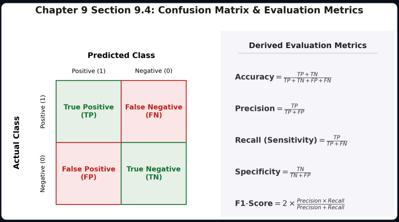
	ทำนายว่าเป็น Positive (Predicted Positive)	ทำนายว่าเป็น Negative (Predicted Negative)
ค่าจริงเป็น Positive (Actual Positive)	True Positive (TP)	False Negative (FN)

ค่าจริงเป็น Negative (Actual Negative)	False Positive (FP)	True Negative (TN)

องค์ประกอบทั้ง 4 ของ Confusion Matrix มีความหมายดังนี้:

1. **True Positive (TP):** ผลบวกจริง คือกรณีที่ข้อมูลจริงเป็น Positive และโมเดลทำนายถูกต้องว่าเป็น Positive
2. **True Negative (TN):** ผลลบจริง คือกรณีที่ข้อมูลจริงเป็น Negative และโมเดลทำนายถูกต้องว่าเป็น Negative
3. **False Positive (FP):** ผลบวกลวง (Type I Error) คือกรณีที่ข้อมูลจริงเป็น Negative แต่โมเดลทำนายผิดว่าเป็น Positive
4. **False Negative (FN):** ผลลบลวง (Type II Error) คือกรณีที่ข้อมูลจริงเป็น Positive แต่โมเดลทำนายผิดว่าเป็น Negative

ตัวอย่างในบริบทการแพทย์: พิจารณาระบบปัญญาประดิษฐ์ของโรงพยาบาลศรีนครินทร์ ที่ใช้ภาพถ่ายรังสีเพื่อวินิจฉัยโรคมะเร็งปอด (Positive = เป็นมะเร็ง, Negative = ปกติ) * **TP:** ผู้ป่วยเป็นมะเร็ง และโมเดลทำนายว่าเป็นมะเร็ง (ผู้ป่วยได้รับการรักษา) * **TN:** ผู้ป่วยปกติ และโมเดลทำนายว่าปกติ (ผู้ป่วยกลับบ้านได้อย่างสบายใจ) * **FP (Type I Error):** ผู้ป่วยปกติ แต่โมเดลทำนายว่าเป็นมะเร็ง (ผู้ป่วยตกใจ ต้องตรวจชิ้นเนื้อเพิ่มเติม เสียค่าใช้จ่าย) * **FN (Type II Error):** ผู้ป่วยเป็นมะเร็ง แต่โมเดลทำนายว่าปกติ (ผู้ป่วยไม่ได้รับการรักษา ร้ายแรงถึงชีวิต)


Chapter 9 Section 9.4: Confusion Matrix & Evaluation Metrics
Predicted Class
Positive (1)
Negative (0)
Actual Class
Positive (1)
Negative (0)
True Positive (TP)
False Negative (FN)
False Positive (FP)
True Negative (TN)
Derived Evaluation Metrics
Accuracy = $\frac{TP+TN}{TP+TN+FP+FN}$
Precision = $\frac{TP}{TP+FP}$
Recall (Sensitivity) = $\frac{TP}{TP+FN}$
Specificity = $\frac{TN}{TN+FP}$
F1-Score = $2 \times \frac{Precision \times Recall}{Precision + Recall}$

Confusion Matrix & Evaluation Metrics

ตัวชี้วัดประสิทธิภาพพื้นฐาน (Basic Performance Metrics)

จากค่า TP, TN, FP และ FN ใน Confusion Matrix สามารถนำมาคำนวณตัวชี้วัดประสิทธิภาพต่างๆ ได้ดังนี้ (VanderPlas, 2016)

ความแม่นยำ (Accuracy)

Accuracy คือสัดส่วนของการทำนายที่ถูกต้องทั้งหมดต่อจำนวนข้อมูลทั้งหมด เป็นตัวชี้วัดที่เข้าใจง่ายที่สุด

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

ข้อควรระวัง: Accuracy เป็นตัวชี้วัดที่ดีเมื่อข้อมูลแต่ละคลาสมีจำนวนใกล้เคียงกัน (Balanced Data) แต่จะให้ภาพที่บิดเบือนอย่างรุนแรงหากข้อมูลมีความไม่สมดุล (Imbalanced Data)

ความเที่ยงตรง (Precision)

Precision คือสัดส่วนของการทำนายว่าเป็น Positive ที่ถูกต้อง ต่อจำนวนที่โมเดลทำนายว่าเป็น Positive ทั้งหมด ตัวชี้วัดนี้ตอบคำถามว่า *“จากทั้งหมดที่โมเดลบอกว่าเป็น Positive มีสิ่งที่เป็น Positive จริงๆ อยู่เท่าใด?”*

$$Precision = \frac{TP}{TP + FP}$$

Precision จะมีค่าสูงเมื่อโมเดลมี **False Positive (FP) ต่ำ**

ความไว หรือ ความระลึกได้ (Recall หรือ Sensitivity)

Recall คือสัดส่วนของการทำนายว่าเป็น Positive ที่ถูกต้อง ต่อจำนวนข้อมูลที่เป็น Positive จริงๆ ทั้งหมดในชุดข้อมูล ตัวชี้วัดนี้ตอบคำถามว่า *“จากข้อมูลที่เป็น Positive ทั้งหมด โมเดลสามารถค้นพบและจับมันได้มากน้อยเพียงใด?”*

$$Recall = \frac{TP}{TP + FN}$$

Recall จะมีค่าสูงเมื่อโมเดลมี **False Negative (FN) ต่ำ**

F1-Score

ในหลายกรณี การเพิ่ม Precision จะทำให้ Recall ลดลง และการเพิ่ม Recall จะทำให้ Precision ลดลง (Precision-Recall Tradeoff) F1-Score ถูกสร้างขึ้นมาเพื่อหาจุดสมดุลระหว่างสองค่านี้ โดยใช้ค่าเฉลี่ยฮาร์มอนิก (Harmonic Mean) แทนค่าเฉลี่ยเลขคณิต (Arithmetic Mean) เพื่อลงโทษโมเดลที่มีค่าใดค่าหนึ่งต่ำเกินไป

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

การเลือกตัวชี้วัดให้เหมาะสมกับบริบท (Choosing Metrics by Context)

การเลือกตัวชี้วัดหลักเพื่อประเมินและปรับแต่ง (Tune) โมเดลนั้น ไม่มีกฎตายตัว แต่ขึ้นอยู่กับบริบทของปัญหา (Business Context) โดยพิจารณาว่าระหว่าง False Positive (FP) และ False Negative (FN) สิ่งใดสร้างความเสียหายมากกว่ากัน

กรณีที่ควรเน้น Recall สูง (ยอมรับ FP ได้ แต่ห้ามเกิด FN)

ใช้เมื่อ **False Negative มีต้นทุนสูงมากหรืออันตรายถึงชีวิต** * **การคัดกรองโรคระบาด (Medical Diagnosis):** เช่น การตรวจหาเชื้อ COVID-19 หากโมเดลทำนายว่าเป็นลบทั้งที่ผู้ป่วยติดเชื้อ (FN) ผู้ป่วยจะนำเชื้อไปแพร่กระจายในชุมชน ซึ่งอันตรายกว่าการที่คนปกติถูกทำนายว่าติดเชื้อ (FP) แล้วต้องไปกักตัวหรือตรวจซ้ำ ดังนั้นระบบนี้ต้องมี Recall สูง * **ระบบตรวจจับผู้บุกรุกเครือข่ายของมหาวิทยาลัย (Intrusion Detection):** หากแฮกเกอร์เจาะระบบได้แต่โมเดลมองว่าเป็นผู้ใช้ปกติ (FN) ข้อมูลของมหาวิทยาลัยจะสูญหาย การแจ้งเตือนผิดพลาดว่าแอดมินเป็นแฮกเกอร์ (FP) ยังถือเป็นเรื่องที่ยอมรับได้มากกว่า

กรณีที่ควรเน้น Precision สูง (ยอมรับ FN ได้ แต่ห้ามเกิด FP)

ใช้เมื่อ **False Positive สร้างความรำคาญหรือทำลายความน่าเชื่อถือของระบบอย่างรุนแรง** * **ระบบกรองอีเมลขยะ (Spam Email Detection):** หากอีเมลสำคัญจากอาจารย์ที่ปรึกษา (Negative) ถูกโมเดลมองว่าเป็นสแปม (Positive) แล้วลบทิ้ง (FP) นักศึกษาอาจพลาดการสอบสำคัญ ในขณะที่การมีอีเมลโฆษณาหลุดรอดมาใน Inbox บ้าง (FN) เป็นเพียงความรำคาญเล็กน้อย * **ระบบแนะนำภาพยนตร์หรือสินค้า (E-commerce Recommendation):** หากระบบแนะนำสินค้าที่ลูกค้าไม่สนใจ (FP) บ่อยครั้ง ลูกค้าจะเลิกเชื่อถือระบบ การแนะนำเฉพาะสิ่งที่มั่นใจจริงๆ ว่าลูกค้าชอบ (Precision สูง) จึงสำคัญกว่าการพยายามแนะนำทุกอย่างที่ลูกค้าอาจจะชอบ (Recall สูง)

ปัญหาข้อมูลที่ไม่สมดุล (Imbalanced Data)

ในโลกแห่งความเป็นจริง ชุดข้อมูลมักไม่ได้มีสัดส่วนคลาสที่เท่ากัน (50:50) ปรากฏการณ์ที่คลาสใดคลาสหนึ่งมีจำนวนมากกว่าอีกคลาสหนึ่งอย่างมหาศาล เรียกว่า **Imbalanced Data** (McKinney, 2022)

พิจารณาข้อมูลธุรกรรมบัตรเครดิตของธนาคารพาณิชย์ในประเทศไทย สมมติว่ามีธุรกรรมปกติ 999,000 รายการ และมีการทุจริต (Fraud) เพียง 1,000 รายการ (สัดส่วน 99.9% ต่อ 0.1%)

หากเราสร้าง “โมเดลไร้ประสิทธิภาพ” ที่ทำนายว่า **“ทุกธุรกรรมเป็นปกติ”** เสมอ โดยไม่สนใจข้อมูลใดๆ เลย: * ทำนายถูกต้อง (TN) = 999,000 รายการ * ทำนายผิด (FN) = 1,000 รายการ * Accuracy = 999,000 / 1,000,000 = **99.9%**

แม้ Accuracy จะสูงถึง 99.9% แต่โมเดลนี้ไร้ประโยชน์โดยสิ้นเชิง เพราะไม่สามารถจับการทุจริตได้เลยแม้แต่รายการเดียว (Recall สำหรับคลาส Fraud = 0%) นี่คือเหตุผลที่นักวิทยาศาสตร์ข้อมูลต้องใช้ Precision, Recall และ F1-Score ควบคู่กับ Confusion Matrix เสมอเมื่อเผชิญกับ Imbalanced Data

การประมวลผลด้วย Python (Scikit-Learn)

โค้ดด้านล่างแสดงการสร้างข้อมูล Imbalanced Data จำลอง และการใช้งานไลบรารี `scikit-learn` เพื่อคำนวณ Metrics ต่างๆ

```
import numpy as np
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score, classi

# 1. สร้างชุดข้อมูลจำลองที่มีปัญหา Imbalanced Data
# สมมติเป็นข้อมูลบัตรเครดิต 10,000 รายการ มีทุจริต (Class 1) เพียง 2%
X, y = make_classification(n_samples=10000, n_features=10,
                           weights=[0.98, 0.02], # กำหนดสัดส่วน 98% และ 2%
                           random_state=42)

# แบ่งข้อมูลเป็นชุดฝึกสอน (Train) 70% และชุดทดสอบ (Test) 30%
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# 2. สร้างและฝึกสอนโมเดล Logistic Regression
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# 3. ให้โมเดลทำนายผลจากชุดข้อมูลทดสอบ
y_pred = model.predict(X_test)

# 4. คำนวณและแสดงผล Metrics ต่างๆ
print("--- Confusion Matrix ---")
cm = confusion_matrix(y_test, y_pred)
print(pd.DataFrame(cm,
                   columns=['Predicted Normal (0)', 'Predicted Fraud (1)'],
                   index=['Actual Normal (0)', 'Actual Fraud (1)']))

print("--- Performance Metrics ---")
print(f"Accuracy:  {accuracy_score(y_test, y_pred):.4f}")
# ต้องระบุ pos_label หรือ average ให้ชัดเจน ในที่นี้เราสนใจคลาส 1 (Fraud)
print(f"Precision: {precision_score(y_test, y_pred):.4f}")
print(f"Recall:    {recall_score(y_test, y_pred):.4f}")
print(f"F1-Score:  {f1_score(y_test, y_pred):.4f}")

print("--- Classification Report ---")
# classification_report จะสรุปค่าทุกอย่างแยกตามคลาสให้โดยอัตโนมัติ
print(classification_report(y_test, y_pred))

--- Confusion Matrix ---
                   Predicted Normal (0)  Predicted Fraud (1)
Actual Normal (0)                  2924                    0
Actual Fraud (1)                     71                    5

--- Performance Metrics ---
Accuracy:  0.9763
Precision: 1.0000
Recall:    0.0658
F1-Score:  0.1235

--- Classification Report ---
              precision    recall  f1-score   support

           0       0.98      1.00      0.99      2924
           1       1.00      0.07      0.12        76

    accuracy                           0.98      3000
   macro avg       0.99      0.53      0.56      3000
weighted avg       0.98      0.98      0.97      3000
```

ผลลัพธ์ที่ได้จากการรันโค้ด (โดยประมาณ): จากการรันโค้ด ผู้อ่านจะสังเกตได้ว่าแม้โมเดลจะมีค่า Accuracy สูงมาก (ประมาณ 0.98 หรือ 98%) แต่เมื่อพิจารณาที่ `classification_report` ในบรรทัดของคลาส 1 (Fraud) ค่า Recall จะอยู่ในระดับที่ต่ำมาก แสดงให้เห็นว่าโมเดลปล่อยให้การทุจริตหลุดรอดไปได้จำนวนมาก (False Negative สูง) ซึ่งยืนยันถึงความล้มเหลวของการใช้ Accuracy เพียงอย่างเดียวในการประเมินข้อมูลที่ไม่สมดุล

ตัวชี้วัดสำหรับ Multiclass และ Multilabel Classification

ในหัวข้อก่อนหน้า เราได้พิจารณาตัวชี้วัดประสิทธิภาพของโมเดลจากปัญหา Binary Classification ซึ่งมีเพียง 2 คลาส ได้แก่ Positive และ Negative อย่างไรก็ตาม ในการใช้งานจริง ปัญหา Classification อาจมีมากกว่า 2 คลาส หรือข้อมูลหนึ่งตัวอย่างอาจมีได้หลายป้ายกำกับพร้อมกัน ดังนั้น การเลือกใช้ Performance Metrics จึงต้องพิจารณาให้เหมาะสมกับลักษณะของปัญหา

Metrics สำหรับ Multiclass Classification

Multiclass Classification คือปัญหาที่ตัวแปรเป้าหมายมีมากกว่า 2 คลาส และข้อมูลหนึ่งตัวอย่างจะอยู่ได้เพียงหนึ่งคลาสเท่านั้น เช่น การจำแนกเกรดของนักศึกษาเป็น A, B, C, D หรือ F หรือการจำแนกภาพถ่ายเมล็ดข้าวเป็นข้าวหอมมะลิ 105, ข้าวเหนียว กข6 หรือข้าวไรซ์เบอร์รี่

ในกรณี Multiclass Classification สามารถคำนวณ Precision, Recall และ F1-score แยกตามแต่ละคลาสได้ โดยมองทีละคลาสเป็น Positive และมองคลาสอื่นทั้งหมดเป็น Negative วิธีนี้เรียกว่า One-vs-Rest Evaluation

ตัวอย่างเช่น หากต้องการคำนวณ Recall ของคลาส “A” จะพิจารณาว่า

- Positive คือ ข้อมูลที่เป็นเกรด A
- Negative คือ ข้อมูลที่ไม่ใช่เกรด A เช่น B, C, D และ F

จากนั้นจึงคำนวณค่า Precision, Recall และ F1-score ของคลาส A ได้เช่นเดียวกับ Binary Classification

วิธีการเฉลี่ยค่า Metrics ใน Multiclass Classification

เมื่อมีหลายคลาส เราจำเป็นต้องสรุปค่า Precision, Recall และ F1-score จากหลายคลาสให้เป็นค่าเดียวเพื่อใช้เปรียบเทียบโมเดล วิธีเฉลี่ยที่นิยมใช้มีดังนี้

วิธีเฉลี่ย	ความหมาย	เหมาะกับกรณี

Macro Average	คำนวณ metric แยกแต่ละคลาส แล้วนำมาเฉลี่ยโดยให้ทุกคลาสมีน้ำหนักเท่ากัน	เหมาะเมื่อสนใจประสิทธิภาพของทุกคลาสเท่าเทียมกัน โดยเฉพาะเมื่อคลาสส่วนน้อยมีความสำคัญ
Micro Average	รวมจำนวน TP, FP และ FN จากทุกคลาสก่อน แล้วจึงคำนวณ metric	เหมาะเมื่อสนใจภาพรวมของการทำนายทั้งหมด
Weighted Average	คำนวณ metric แยกแต่ละคลาส แล้วเฉลี่ยโดยถ่วงน้ำหนักตามจำนวนตัวอย่างของแต่ละคลาส	เหมาะเมื่อข้อมูลแต่ละคลาสมีจำนวนไม่เท่ากัน และต้องการค่าเฉลี่ยที่สะท้อนสัดส่วนของข้อมูลจริง

ตัวอย่างการตีความ Macro และ Weighted Average

สมมติว่าโมเดลจำแนกเกรดนักศึกษาเป็น A, B, C, D และ F โดยมีนักศึกษาเกรด A และ B จำนวนมาก แต่มีนักศึกษาเกรด F จำนวนน้อย หากใช้ Weighted F1-score ค่าเฉลี่ยที่ได้อาจสูง เพราะคลาสที่มีจำนวนมากมีอิทธิพลต่อผลลัพธ์มากกว่า ในทางตรงกันข้าม Macro F1-score จะให้ความสำคัญกับทุกคลาสเท่ากัน จึงช่วยสะท้อนว่าโมเดลทำงานได้ดีหรือไม่กับคลาสที่มีจำนวนน้อย เช่น คลาส F

ดังนั้น หากคลาสส่วนน้อยมีความสำคัญ เช่น กลุ่มนักศึกษาที่มีความเสี่ยงสอบตก หรือผู้ป่วยที่มีโรคร้ายแรง ควรพิจารณา Macro Recall หรือ Macro F1-score ควบคู่กับ Weighted F1-score

Metrics สำหรับ Multilabel Classification

Multilabel Classification คือปัญหาที่ข้อมูลหนึ่งตัวอย่างสามารถมีได้หลายป้ายกำกับพร้อมกัน เช่น ข่าวหนึ่งข่าวอาจเกี่ยวข้องกับหลายหมวดหมู่ ได้แก่ “เศรษฐกิจ”, “เทคโนโลยี” และ “การศึกษา” หรือภาพยนตร์หนึ่งเรื่องอาจมีหลายประเภท เช่น Action, Sci-Fi และ Comedy

ใน Multilabel Classification การประเมินผลจะซับซ้อนกว่า Multiclass Classification เพราะโมเดลต้องตัดสินใจแยกกันในแต่ละ label ว่า label นั้นควรถูกเลือกหรือไม่

ตัวชี้วัดที่นิยมใช้ ได้แก่

Metric	ความหมาย	การตีความ
Subset Accuracy	สัดส่วนของตัวอย่างที่โมเดลทำนาย label ได้ถูกต้องทั้งหมดทุก label	เข้มงวดมาก เพราะต้องทำนายทุก label ให้ตรงทั้งหมด
Hamming Loss	สัดส่วนของ label ที่ทำนายผิดเมื่อเทียบกับจำนวน label ทั้งหมด	ค่ายิ่งต่ำยิ่งดี
Jaccard Score	วัดความคล้ายกันระหว่างชุด label ที่ทำนายได้กับชุด label จริง	ค่ายิ่งสูงยิ่งดี
Micro F1-score	รวมผลการทำนายของทุก label ก่อนคำนวณ F1-score	เหมาะเมื่อสนใจภาพรวมของทุก label
Macro F1-score	คำนวณ F1-score แยกแต่ละ label แล้วเฉลี่ยแบบเท่ากัน	เหมาะเมื่อต้องการให้ทุก label มีความสำคัญเท่ากัน
Samples F1-score	คำนวณ F1-score แยกตามแต่ละตัวอย่าง แล้วนำมาเฉลี่ย	เหมาะกับ Multilabel Classification โดยเฉพาะ

ตัวอย่างการใช้ Metrics ใน Python

```
from sklearn.metrics import f1_score, hamming_loss, jaccard_score, classification_report

# แต่ละแถวแทนข้อมูล 1 ตัวอย่าง
# แต่ละคอลัมน์แทน label หนึ่งประเภท
# 1 = มี label นั้น, 0 = ไม่มี label นั้น
#
# Label 0 = Technology
# Label 1 = Economy
# Label 2 = Education

y_true_multi = [
    [1, 0, 1],  # ข่าวที่ 1: Technology, Education
    [0, 1, 0],  # ข่าวที่ 2: Economy
    [1, 1, 0],  # ข่าวที่ 3: Technology, Economy
    [0, 0, 1],  # ข่าวที่ 4: Education
    [1, 1, 1],  # ข่าวที่ 5: Technology, Economy, Education
]

y_pred_multi = [
    [1, 0, 1],  # ทำนายถูกทั้งหมด
    [0, 1, 1],  # ทำนาย Economy ถูก แต่เพิ่ม Education เกินมา
    [1, 0, 0],  # ทำนาย Technology ถูก แต่พลาด Economy
    [0, 0, 1],  # ทำนายถูกทั้งหมด
    [1, 1, 0],  # ทำนาย Technology, Economy ถูก แต่พลาด Education
]

label_names = ["Technology", "Economy", "Education"]

print("--- Classification Report ---")
print(classification_report(
    y_true_multi,
    y_pred_multi,
    target_names=label_names
))

print("Micro F1:", f1_score(y_true_multi, y_pred_multi, average="micro"))
print("Macro F1:", f1_score(y_true_multi, y_pred_multi, average="macro"))
print("Weighted F1:", f1_score(y_true_multi, y_pred_multi, average="weighted"))
print("Samples F1:", f1_score(y_true_multi, y_pred_multi, average="samples"))

print("Hamming Loss:", hamming_loss(y_true_multi, y_pred_multi))
print("Jaccard Score:", jaccard_score(y_true_multi, y_pred_multi, average="samples"))
```

ผลลัพธ์ที่ได้จากการรันโค้ด (โดยประมาณ):

```
--- Classification Report ---
              precision    recall  f1-score   support

  Technology       1.00      1.00      1.00         3
     Economy       1.00      0.67      0.80         3
   Education       0.67      0.67      0.67         3

   micro avg       0.88      0.78      0.82         9
   macro avg       0.89      0.78      0.82         9
weighted avg       0.89      0.78      0.82         9
 samples avg       0.90      0.83      0.83         9

Micro F1: 0.823529411764706
Macro F1: 0.8222222222222223
Weighted F1: 0.8222222222222223
Samples F1: 0.8266666666666665
Hamming Loss: 0.2
Jaccard Score: 0.7333333333333333
```

การตีความผลลัพธ์

จากตัวอย่าง Multilabel Classification ข้างต้น ข้อมูลแต่ละตัวอย่างสามารถมีได้หลาย label พร้อมกัน ดังนั้นการประเมินผลจึงไม่ใช่เพียงดูว่า “ทำนายคลาสถูกหรือผิด” เท่านั้น แต่ต้องพิจารณาว่าโมเดลทำนาย label แต่ละตัวถูกต้องมากน้อยเพียงใด

ค่า `Micro F1` เท่ากับประมาณ `0.82` หมายถึง เมื่อรวมการทำนายของทุก label เข้าด้วยกัน โมเดลมีสมดุลระหว่าง Precision และ Recall อยู่ในระดับค่อนข้างดี

ค่า `Macro F1` เท่ากับประมาณ `0.82` เช่นกัน แสดงว่าเมื่อเฉลี่ยผลลัพธ์ของแต่ละ label โดยให้น้ำหนักเท่ากัน โมเดลยังทำงานได้ดีในภาพรวม

ค่า `Samples F1` ใช้เฉพาะกับ Multilabel Classification โดยคำนวณ F1-score แยกในแต่ละตัวอย่าง แล้วนำมาเฉลี่ย เหมาะสำหรับกรณีที่ต้องการดูว่าโมเดลทำนายชุด label ของแต่ละข้อมูลได้ดีเพียงใด

ค่า `Hamming Loss` เท่ากับ `0.2` หมายความว่า จากการตัดสินใจทั้งหมดในระดับ label มีสัดส่วนที่ทำนายผิดประมาณ 20% โดยค่านี้ยิ่งต่ำยิ่งดี

ค่า `Jaccard Score` เท่ากับประมาณ `0.73` หมายถึง ชุด label ที่โมเดลทำนายได้มีความคล้ายกับชุด label จริงประมาณ 73% โดยค่านี้ยิ่งสูงยิ่งดี

แนวทางการเลือก Metrics

ลักษณะปัญหา	Metric ที่ควรพิจารณา
Binary Classification และข้อมูลสมดุล	Accuracy, Precision, Recall, F1-score
Binary Classification และข้อมูลไม่สมดุล	Precision, Recall, F1-score, ROC-AUC, PR-AUC
Multiclass Classification และข้อมูลสมดุล	Accuracy, Macro F1-score
Multiclass Classification และข้อมูลไม่สมดุล	Macro F1-score, Weighted F1-score, Per-class Recall
Multilabel Classification	Micro F1-score, Macro F1-score, Samples F1-score, Hamming Loss, Jaccard Score
งานทางการแพทย์หรือความปลอดภัย	Recall, Sensitivity, Macro Recall
งานที่ต้องลดการแจ้งเตือนผิดพลาด	Precision

สรุปประเด็นสำคัญ

- **Confusion Matrix** เป็นเครื่องมือพื้นฐานที่แยกแยะความผิดพลาดออกเป็น False Positive (Type I Error) และ False Negative (Type II Error) ช่วยให้เห็นภาพรวมของการทำนายอย่างละเอียด
- **Accuracy** เป็นตัวชี้วัดที่หลอกตาได้ง่ายหากนำไปใช้กับข้อมูลที่มีความไม่สมดุล (Imbalanced Data)
- **Precision** ควรเป็นตัวชี้วัดหลักเมื่อต้นทุนของ False Positive สูง (เช่น ระบบกรองอีเมลขยะ)
- **Recall** ควรเป็นตัวชี้วัดหลักเมื่อต้นทุนของ False Negative สูง (เช่น การวินิจฉัยโรคร้ายแรง)
- **F1-Score** เป็นค่าเฉลี่ยฮาร์มอนิกระหว่าง Precision และ Recall เหมาะสำหรับใช้ประเมินโมเดลในภาพรวมเมื่อข้อมูลมีความไม่สมดุล
- **Multiclass Classification** ค่า `classification_report` จะรายงานผลแยกตามแต่ละคลาส และสรุปด้วย `accuracy`, `macro avg` และ `weighted avg`
- **Multilabel Classification** ค่า `classification_report` จะรายงานผลแยกตามแต่ละ label และเพิ่มค่าเฉลี่ยแบบ `micro avg`, `macro avg`, `weighted avg` และ `samples avg` ดังนั้น เมื่อตีความผลลัพธ์ ควรพิจารณาทั้งค่ารายคลาสหรือราย label และค่าเฉลี่ยรวม โดยเฉพาะในกรณีที่ข้อมูลมีความไม่สมดุลหรือมีบางคลาสที่มีความสำคัญเป็นพิเศษ
- **การเลือก Metrics** ควรพิจารณาจากลักษณะของปัญหา จำนวนคลาส ความไม่สมดุลของข้อมูล และต้นทุนของความผิดพลาดแต่ละประเภทประกอบกันเสมอ

คำถามท้าย Section

1. พิจารณาระบบอนุมัติสินเชื่อของธนาคารพาณิชย์ หากโมเดลทำนายว่า “ลูกค้าจะผิดนัดชำระหนี้ (Positive)” ความผิดพลาดแบบ False Positive และ False Negative ในบริบทนี้คืออะไร และในฐานะผู้บริหารธนาคาร ท่านจะเลือกปรับจูนโมเดลให้เน้นค่า Precision หรือ Recall เพราะเหตุใด?
2. จาก Confusion Matrix ต่อไปนี้ มีข้อมูลทั้งหมดกี่รายการ และจงคำนวณหาค่า Accuracy, Precision, Recall และ F1-Score สำหรับคลาส Positive
    - TP = 40, TN = 900, FP = 10, FN = 50
3. อธิบายสาเหตุทางคณิตศาสตร์ว่าเหตุใด F1-Score จึงใช้ค่าเฉลี่ยฮาร์มอนิก (Harmonic Mean) แทนการใช้ค่าเฉลี่ยเลขคณิต (Arithmetic Mean) แบบปกติในการหาจุดสมดุลระหว่าง Precision และ Recall?

การเปรียบเทียบและเลือก Model

ในการพัฒนาระบบ Machine Learning สำหรับการจำแนกประเภท (Classification) การเลือกใช้อัลกอริทึมที่เหมาะสมกับชุดข้อมูลและบริบทของปัญหาถือเป็นขั้นตอนที่มีความสำคัญอย่างยิ่ง ตามทฤษฎี No Free Lunch Theorem (Wolpert, 1996) ไม่มีอัลกอริทึมใดที่มีประสิทธิภาพสูงสุดสำหรับทุกปัญหา ดังนั้น วิศวกรข้อมูลและนักวิทยาศาสตร์ข้อมูลจำเป็นต้องมีความเข้าใจอย่างลึกซึ้งถึงกลไกการทำงาน ข้อดี ข้อเสีย และข้อจำกัดของแต่ละ Model เพื่อนำมาเปรียบเทียบและคัดเลือกได้อย่างเป็นระบบ

ในหัวข้อนี้ จะทำการศึกษาเปรียบเทียบระหว่างสองอัลกอริทึมพื้นฐานที่มีลักษณะการทำงานแตกต่างกันอย่างสิ้นเชิง ได้แก่ Decision Tree (DT) และ k-Nearest Neighbors (k-NN) รวมถึงการศึกษาปรากฏการณ์ Overfitting ผ่านการปรับแต่งค่าพารามิเตอร์ (Hyperparameter Tuning) และการคัดเลือกคุณลักษณะ (Feature Selection) เพื่อเพิ่มประสิทธิภาพของ Model

การเปรียบเทียบระหว่าง Decision Tree และ k-Nearest Neighbors (When to use which)

Decision Tree และ k-Nearest Neighbors จัดเป็นอัลกอริทึมประเภท Supervised Learning ที่สามารถใช้ในงาน Classification ได้ทั้งคู่ อย่างไรก็ตาม โครงสร้างทางคณิตศาสตร์และแนวคิดเบื้องหลังมีความแตกต่างกันอย่างชัดเจน ซึ่งส่งผลต่อการนำไปประยุกต์ใช้ในสถานการณ์จริง

1. กลไกการเรียนรู้ (Learning Mechanism) * **k-Nearest Neighbors (k-NN):** เป็นอัลกอริทึมประเภท Instance-based learning หรือ Lazy learning (James et al., 2023) k-NN ไม่ได้สร้างสมการหรือโครงสร้าง Model ในช่วงการฝึกสอน (Training phase) แต่จะจดจำข้อมูลฝึกสอนทั้งหมดไว้ เมื่อต้องการทำนายข้อมูลใหม่ k-NN จะคำนวณระยะห่าง (Distance metric เช่น Euclidean หรือ Manhattan) ระหว่างข้อมูลใหม่กับข้อมูลฝึกสอนทั้งหมด เพื่อหาเพื่อนบ้านที่ใกล้ที่สุดจำนวน *k* ตัว และทำการลงคะแนนเสียง (Majority voting) * **Decision Tree (DT):** เป็นอัลกอริทึมประเภท Eager learning โดยจะสร้างโครงสร้างต้นไม้การตัดสินใจในช่วงการฝึกสอน ผ่านการคำนวณค่า Information Gain หรือ Gini Impurity เพื่อเลือกคุณลักษณะ (Feature) ที่สามารถแบ่งแยกกลุ่มข้อมูลได้ดีที่สุดในแต่ละโหนด (Node) (VanderPlas, 2016) เมื่อสร้างต้นไม้เสร็จสิ้น Model จะมีกฎการตัดสินใจ (Decision rules) ที่ชัดเจนในรูปแบบ If-Then-Else

2. การเตรียมข้อมูล (Data Preparation) * **k-NN:** มีความอ่อนไหวต่อขนาดของข้อมูล (Scale-sensitive) อย่างมาก เนื่องจากใช้การคำนวณระยะห่าง หากคุณลักษณะหนึ่งมีหน่วยเป็นหลักล้าน (เช่น รายได้) และอีกคุณลักษณะมีหน่วยเป็นหลักหน่วย (เช่น อายุ) ระยะห่างจะถูกครอบงำโดยคุณลักษณะที่มีค่าขนาดใหญ่ ดังนั้น **จำเป็นต้องทำ Feature Scaling** (เช่น Standardization หรือ Normalization) ก่อนเสมอ นอกจากนี้ k-NN ยังอ่อนไหวต่อข้อมูลที่มีความผิดปกติ (Outliers) * **Decision Tree:** ไม่สนใจขนาดของข้อมูล (Scale-invariant) เนื่องจากการแบ่งโหนดพิจารณาจากการเรียงลำดับค่าและการหาจุดตัด (Threshold) ที่ดีที่สุดในแต่ละคุณลักษณะแยกจากกัน จึง **ไม่จำเป็นต้องทำ Feature Scaling** และมีความทนทานต่อ Outliers มากกว่า

3. ความสามารถในการอธิบายผล (Interpretability) * **k-NN:** มีความสามารถในการอธิบายผลในระดับต่ำ (Low interpretability) หรือเป็นเสมือน Black-box ในกรณีที่มีมิติข้อมูลจำนวนมาก การอธิบายว่าเหตุใดข้อมูลบรรทัดนี้จึงถูกจัดอยู่ในคลาส A ทำได้เพียงอ้างอิงว่า “เพราะมีความคล้ายคลึงกับข้อมูลในอดีต *k* ตัว” * **Decision Tree:** มีความสามารถในการอธิบายผลในระดับสูงมาก (High interpretability) หรือเป็น White-box Model สามารถแสดงผลในรูปแบบกราฟิกที่มนุษย์เข้าใจได้ทันที ทำให้เหมาะกับงานที่ต้องการเหตุผลประกอบการตัดสินใจทางกฎหมายหรือธุรกิจ

ตารางที่ 9.1 สรุปการเปรียบเทียบระหว่าง Decision Tree และ k-NN

คุณสมบัติ	k-Nearest Neighbors (k-NN)	Decision Tree (DT)
ประเภทการเรียนรู้	Lazy Learning	Eager Learning
ระยะเวลาในการ Train	รวดเร็วมาก (เพียงแค่จัดเก็บข้อมูล)	ใช้เวลาพอสมควร (ต้องคำนวณจุดตัดทุก Feature)
ระยะเวลาในการ Predict	ช้า (ต้องคำนวณระยะห่างกับข้อมูลทุกตัว)	รวดเร็วมาก (วิ่งตามเงื่อนไขจาก Root สู่ Leaf)
ความจำเป็นในการ Scale ข้อมูล	จำเป็นอย่างยิ่ง	ไม่จำเป็น
ความสามารถในการอธิบายผล	ต่ำ	สูงมาก
การจัดการกับ Missing Values	ไม่สามารถจัดการได้โดยตรง	บาง Implementation สามารถจัดการได้

ตัวอย่างการเลือกใช้ในบริบทประเทศไทย: * **กรณีที่ควรใช้ Decision Tree:** ระบบพิจารณาอนุมัติสินเชื่อของธนาคารพาณิชย์ในประเทศไทย เนื่องจากธนาคารแห่งประเทศไทย (ธปท.) มีข้อกำหนดให้สถาบันการเงินต้องสามารถชี้แจงเหตุผลในการปฏิเสธสินเชื่อแก่ลูกค้าได้ (Explainable AI) Decision Tree สามารถระบุได้ชัดเจนว่า “ปฏิเสธสินเชื่อเนื่องจาก รายได้ < 15,000 บาท และมีประวัติค้างชำระ > 2 ครั้ง” * **กรณีที่ควรใช้ k-NN:** ระบบแนะนำสินค้า (Recommendation System) บนแพลตฟอร์ม e-commerce หรือระบบจับคู่กลุ่มผู้ป่วยที่มีอาการคล้ายคลึงกันในโรงพยาบาลศูนย์ (เช่น โรงพยาบาลศรีนครินทร์) เพื่อวิเคราะห์แนวทางการรักษาที่เคยใช้ได้ผลกับผู้ป่วยที่มีลักษณะทางกายภาพและผลเลือดใกล้เคียงกันที่สุด (Nearest neighbors)

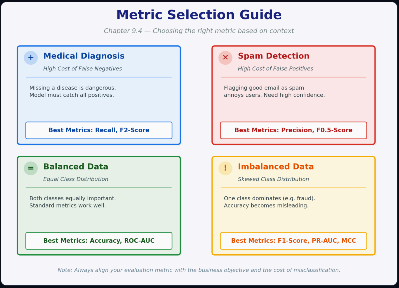
ปรากฏการณ์ Overfitting และการปรับแต่งค่าความลึกของต้นไม้ (Depth Tuning)

ปัญหาที่พบได้บ่อยที่สุดในการใช้ Decision Tree คือ **ปรากฏการณ์ Overfitting (การเรียนรู้มากเกินไป)** ซึ่งเกิดขึ้นเมื่อ Model พยายามสร้างเงื่อนไขการแตกกิ่ง (Splits) เพื่อให้สามารถจำแนกข้อมูลฝึกสอน (Training Data) ได้ถูกต้องสมบูรณ์ 100% ส่งผลให้ต้นไม้มีความลึกมากและมีกิ่งก้านที่ซับซ้อน (High Variance) Model จะจดจำแม้กระทั่งสัญญาณรบกวน (Noise) หรือรายละเอียดเฉพาะตัวของข้อมูลฝึกสอน

พิจารณาบริบทของระบบทำนายการตกออก (Dropout rate) ของนักศึกษามหาวิทยาลัยขอนแก่น หาก Decision Tree เกิด Overfitting โครงสร้างต้นไม้อาจสร้างกฎเกณฑ์ที่เฉพาะเจาะจงเกินไป เช่น “ถ้านักศึกษามาจากจังหวัด A และอยู่หอพัก B และได้เกรดวิชาแคลคูลัส 1.5 จะตกออก” ซึ่งกฎนี้อาจบังเอิญตรงกับนักศึกษาเพียง 1 คนในชุดข้อมูลฝึกสอน แต่ไม่สามารถนำไปใช้ทำนาย (Generalize) กับนักศึกษารุ่นถัดไปได้อย่างมีประสิทธิภาพ (Test Error สูง)

เพื่อป้องกันปัญหานี้ เราจำเป็นต้องทำ **Hyperparameter Tuning** โดยพารามิเตอร์ที่สำคัญที่สุดของ Decision Tree คือ `max_depth` (ความลึกสูงสุดของต้นไม้) การจำกัดความลึกเปรียบเสมือนการทำ Regularization รูปแบบหนึ่ง เพื่อบังคับให้ Model เรียนรู้เฉพาะรูปแบบที่สำคัญจริงๆ

```
แผนภาพแสดงความสัมพันธ์ระหว่าง ความซับซ้อนของ Model (max_depth) และค่าความผิดพลาด (Error Rate)
(Bias-Variance Tradeoff)

Error Rate
  |
  |       Training Error (ลดลงเรื่อยๆ จนเข้าใกล้ 0 เมื่อต้นไม้ลึกขึ้น)
  |      \
  |       \       Test Error (ลดลงในช่วงแรก แต่กลับเพิ่มขึ้นเมื่อเกิด Overfitting)
  |        \     /
  |         \   /  ⟵ จุดเริ่มต้นของ Overfitting
  |          \ /
  |           / \
  |          /   \
  |         /     \
  +--------|------------------------- ความลึกของต้นไม้ (max_depth)
       จุดที่เหมาะสม (Optimal Depth)
```


Metric Selection Guide

Chapter 9.4 — Choosing the right metric based on context

Medical Diagnosis
High Cost of False Negatives
Missing a disease is dangerous.
Model must catch all positives.
Best Metrics: Recall, F2-Score

Spam Detection
High Cost of False Positives
Flagging good email as spam
annoys users. Need high confidence.
Best Metrics: Precision, F0.5-Score

Balanced Data
Equal Class Distribution
Both classes equally important.
Standard metrics work well.
Best Metrics: Accuracy, ROC-AUC

Imbalanced Data
Skewed Class Distribution
One class dominates (e.g. fraud).
Accuracy becomes misleading.
Best Metrics: F1-Score, PR-AUC, MCC

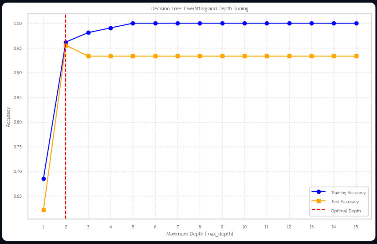
Note: Always align your evaluation metric with the business objective and the cost of misclassification.

Metrics in Context

ตัวอย่างโค้ด: การสาธิต Overfitting และ Depth Tuning ด้วย Python

ในตัวอย่างนี้ จะใช้ชุดข้อมูลมาตรฐาน (Iris Dataset) เพื่อแสดงให้เห็นถึงการเปลี่ยนแปลงของค่าความแม่นยำ (Accuracy) เมื่อปรับค่า `max_depth`

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 1. โหลดชุดข้อมูล Iris
iris = load_iris()
X = iris.data
y = iris.target

# 2. แบ่งข้อมูลเป็น Training Set (70%) และ Test Set (30%)
# กำหนด random_state เพื่อให้ผลการทดลองสามารถทำซ้ำได้ (Reproducible)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=123)

# กำหนดช่วงของความลึกที่ต้องการทดสอบ (ตั้งแต่ 1 ถึง 15)
max_depths = range(1, 16)
train_accuracies = []
test_accuracies = []

# 3. วนลูปเพื่อฝึกสอน Model ด้วย max_depth ที่แตกต่างกัน
for depth in max_depths:
    # สร้าง Model Decision Tree โดยกำหนดความลึกสูงสุด
    clf = DecisionTreeClassifier(max_depth=depth, random_state=1234)
    clf.fit(X_train, y_train)

    # ทำนายผลและคำนวณความแม่นยำสำหรับชุดข้อมูลฝึกสอน
    y_train_pred = clf.predict(X_train)
    train_acc = accuracy_score(y_train, y_train_pred)
    train_accuracies.append(train_acc)

    # ทำนายผลและคำนวณความแม่นยำสำหรับชุดข้อมูลทดสอบ
    y_test_pred = clf.predict(X_test)
    test_acc = accuracy_score(y_test, y_test_pred)
    test_accuracies.append(test_acc)

# 4. สร้างกราฟเปรียบเทียบเพื่อวิเคราะห์ Overfitting
plt.figure(figsize=(10, 6))
plt.plot(max_depths, train_accuracies, label='Training Accuracy', marker='o', color='blue')
plt.plot(max_depths, test_accuracies, label='Test Accuracy', marker='s', color='orange')
plt.title('Decision Tree: Overfitting and Depth Tuning')
plt.xlabel('Maximum Depth (max_depth)')
plt.ylabel('Accuracy')
plt.xticks(max_depths)
plt.axvline(x=2, color='red', linestyle='--', label='Optimal Depth')
plt.legend()
plt.grid(True, linestyle=':', alpha=0.7)
plt.show()
```



alt text

การวิเคราะห์ผลลัพธ์จากกราฟ: จากการรันโค้ดข้างต้น จะพบว่าเมื่อ `max_depth = 1` หรือ 2 ค่า Accuracy ทั้งบนชุดฝึกสอนและชุดทดสอบยังอยู่ในระดับต่ำ เรียกว่าเกิดปรากฏการณ์ Underfitting (Model ง่ายเกินไป) เมื่อเพิ่ม `max_depth` เป็น 2 ค่า Test Accuracy จะขึ้นไปแตะจุดสูงสุด (ประมาณ 96% สำหรับชุดข้อมูลนี้) แต่เมื่อเพิ่ม `max_depth` ตั้งแต่ 5 เป็นต้นไป Training Accuracy จะพุ่งเข้าสู่ 1.0 (100%) เนื่องจาก Model ท่องจำข้อมูลฝึกสอนได้ทั้งหมด ในขณะที่ Test Accuracy จะเริ่มคงที่หรือตกลงเล็กน้อย นี่คือสัญญาณที่ชัดเจนของ **Overfitting** ดังนั้นจากการทดสอบนี้ ค่า `max_depth` ที่เหมาะสมที่สุด (Optimal Depth) สำหรับชุดข้อมูลนี้คือ 3 (McKinney, 2022)

การคัดเลือกคุณลักษณะ (Feature Selection) และกรณีศึกษา Petal-only Decision Tree

ในยุคของ Big Data ชุดข้อมูลมักประกอบไปด้วยคุณลักษณะ (Features) จำนวนมหาศาล ซึ่งบางคุณลักษณะอาจไม่มีส่วนเกี่ยวข้องกับตัวแปรเป้าหมาย (Target variable) เลย หรืออาจมีข้อมูลที่เป็นสัญญาณรบกวน (Noise) การนำคุณลักษณะทั้งหมดเข้าสู่ Model ส่งผลเสียหลายประการ เรียกว่า **Curse of Dimensionality** (คำสาปของมิติข้อมูล) ได้แก่: 1. เพิ่มระยะเวลาและทรัพยากรในการประมวลผล 2. เพิ่มความเสี่ยงในการเกิด Overfitting 3. ลดความสามารถในการอธิบายผล (Interpretability) เนื่องจาก Model มีความซับซ้อนเกินความจำเป็น

Feature Selection คือกระบวนการคัดเลือกเฉพาะคุณลักษณะที่มีอิทธิพลสูงต่อการจำแนกคลาส โดยอัลกอริทึม Decision Tree มีข้อได้เปรียบประการหนึ่งคือ มีกลไกการคัดเลือกคุณลักษณะแฝงอยู่ในตัว (Embedded Feature Selection) ในระหว่างการสร้างต้นไม้ หากคุณลักษณะใดไม่สามารถช่วยลดค่าความไม่บริสุทธิ์ (Impurity) ของข้อมูลได้ คุณลักษณะนั้นจะไม่ถูกเลือกมาเป็นโหนดตัดสินใจ

กรณีศึกษา: ชุดข้อมูล Iris (Petal-only DT) ชุดข้อมูลดอกไอริสประกอบด้วย 4 คุณลักษณะ ได้แก่ ความยาวกลีบเลี้ยง (Sepal Length), ความกว้างกลีบเลี้ยง (Sepal Width), ความยาวกลีบดอก (Petal Length) และ ความกว้างกลีบดอก (Petal Width) จากการศึกษาทางพฤกษศาสตร์และการวิเคราะห์ข้อมูลเชิงสำรวจ (Exploratory Data Analysis) พบว่า ขนาดของ “กลีบดอก (Petal)” มีความสามารถในการจำแนกสายพันธุ์ดอกไอริสได้ดีกว่า “กลีบเลี้ยง (Sepal)” อย่างมีนัยสำคัญ เราจะทำการทดสอบสร้าง Decision Tree โดยใช้ข้อมูลเพียง 2 มิติ (Petal Length และ Petal Width) เพื่อดูว่าประสิทธิภาพของ Model จะลดลงหรือไม่เมื่อเทียบกับการใช้ทั้ง 4 มิติ

```
import pandas as pd
from sklearn.tree import plot_tree

# 1. สร้าง Decision Tree โดยใช้ข้อมูลทั้ง 4 Features (เพื่อดู Feature Importance)
clf_all = DecisionTreeClassifier(max_depth=3, random_state=42)
clf_all.fit(X_train, y_train)

# แสดงค่าความสำคัญของคุณลักษณะ (Feature Importances)
feature_names = iris.feature_names
importances = clf_all.feature_importances_
```

```
print("--- ค่าความสำคัญของคุณลักษณะ (Feature Importances) ---")
for name, imp in zip(feature_names, importances):
    print(f"{name}: {imp:.4f}")

# ผลลัพธ์ที่ได้จะชี้ชัดว่า Petal Length และ Petal Width มีค่าความสำคัญสูงเกือบ 100%
# ในขณะที่ Sepal Features มีค่าเข้าใกล้ 0

# 2. คัดเลือกเฉพาะ Petal Features (Index 2 และ 3)
# X[:, 2:] หมายถึงการดึงข้อมูลทุกแถว แต่เอาเฉพาะคอลัมน์ที่ 2 และ 3
X_train_petal = X_train[:, 2:]
X_test_petal = X_test[:, 2:]

# 3. สร้าง Model ใหม่ด้วยข้อมูล Petal-only
clf_petal = DecisionTreeClassifier(max_depth=3, random_state=42)
clf_petal.fit(X_train_petal, y_train)

# 4. ประเมินประสิทธิภาพเปรียบเทียบกัน
y_pred_all = clf_all.predict(X_test)
acc_all = accuracy_score(y_test, y_pred_all)

y_pred_petal = clf_petal.predict(X_test_petal)
acc_petal = accuracy_score(y_test, y_pred_petal)

print(f"Accuracy (ใช้ทั้ง 4 Features): {acc_all:.4f}")
print(f"Accuracy (ใช้เฉพาะ Petal 2 Features): {acc_petal:.4f}")

--- ค่าความสำคัญของคุณลักษณะ (Feature Importances) ---
sepal length (cm): 0.0000
sepal width (cm): 0.0000
petal length (cm): 0.5703
petal width (cm): 0.4297

Accuracy (ใช้ทั้ง 4 Features): 0.9333
Accuracy (ใช้เฉพาะ Petal 2 Features): 0.9333
```

การวิเคราะห์ผลลัพธ์: จากการรันโค้ด จะพบว่าค่า Accuracy ของ Model ที่ใช้เพียง 2 คุณลักษณะ (Petal-only) มีค่าเท่ากับหรือใกล้เคียงกับ Model ที่ใช้ทั้ง 4 คุณลักษณะอย่างมาก การตัดคุณลักษณะ Sepal ออกไม่ทำให้ประสิทธิภาพลดลง แต่กลับทำให้ Model มีขนาดเล็กลง ประมวลผลเร็วขึ้น และสามารถนำไปแสดงผล (Visualize) บนกราฟ 2 มิติได้อย่างชัดเจน ซึ่งเป็นตัวอย่างที่แสดงให้เห็นถึงประโยชน์ของการทำ Feature Selection อย่างเป็นรูปธรรม (Grus, 2019)

การลดมิติข้อมูลในลักษณะนี้มีความสำคัญมากในระบบจริง ตัวอย่างเช่น ในระบบวิเคราะห์ภาพถ่ายทางการแพทย์ (Medical Image Analysis) การคัดเลือกเฉพาะพิกเซล หรือ Feature ระดับสูงที่บ่งชี้ถึงเซลล์มะเร็งได้จริง (Feature Selection) จะช่วยให้แพทย์สามารถวินิจฉัยโรคได้รวดเร็วและแม่นยำกว่าการนำข้อมูลพิกเซลทั้งหมดของรูปภาพไปประมวลผลโดยตรง

สรุปประเด็นสำคัญ

- **การเลือกใช้อัลกอริทึม:** ไม่ควรมองเพียงค่าความแม่นยำ (Accuracy) แต่ต้องพิจารณาบริบทของข้อมูลและธุรกิจ k-NN เหมาะกับงานที่ต้องการหาความคล้ายคลึง แต่ต้องระวังเรื่องการ Scale ข้อมูล ในขณะที่ Decision Tree เหมาะกับงานที่ต้องการความสามารถในการอธิบายผล (Interpretability) สูง
- **Overfitting และ Bias-Variance Tradeoff:** Decision Tree มีแนวโน้มที่จะเกิด Overfitting ได้ง่ายหากปล่อยให้ต้นไม้เติบโตจนสุด การใช้เทคนิค Hyperparameter Tuning เช่น การจำกัด `max_depth` เป็นวิธีที่มีประสิทธิภาพในการป้องกันปัญหานี้
- **Feature Selection:** การคัดกรองเฉพาะคุณลักษณะที่สำคัญ (เช่น การใช้เฉพาะ Petal features ในชุดข้อมูล Iris) ช่วยลดความซับซ้อนของ Model ลดระยะเวลาการประมวลผล และป้องกัน Curse of Dimensionality โดยไม่ทำให้ประสิทธิภาพในการทำนายลดลง

คำถามท้าย Section

1. หากธนาคารพาณิชย์ต้องการพัฒนาระบบ AI เพื่ออนุมัติบัตรเครดิต โดยมีเงื่อนไขว่าต้องสามารถอธิบายเหตุผลให้ลูกค้าทราบได้เมื่อถูกปฏิเสธ นักศึกษาจะเลือกใช้อัลกอริทึมใดระหว่าง k-NN และ Decision Tree จงอธิบายเหตุผลประกอบเชิงวิชาการ
2. จากกราฟแสดงความสัมพันธ์ระหว่าง `max_depth` และ Accuracy หากพบว่าค่า Training Accuracy สูงถึง 99% แต่ค่า Test Accuracy อยู่ที่เพียง 65% ปรากฏการณ์นี้เรียกว่าอะไร และนักศึกษาจะมีแนวทางแก้ไขเบื้องต้นอย่างไร
3. อธิบายกลไกการทำงานของ Decision Tree ว่าเหตุใดจึงสามารถทำหน้าที่เป็น Embedded Feature Selection ได้ในตัว โดยอ้างอิงจากแนวคิดการคำนวณ Information Gain หรือ Gini Impurity

Logistic Regression

ก่อนที่จะศึกษาอัลกอริทึม Classification ที่ซับซ้อน นักศึกษาควรทำความเข้าใจ **Logistic Regression** ซึ่งเป็น Baseline Model ที่ใช้กันมากที่สุดในอุตสาหกรรม เนื่องจากมีเหตุผลสำคัญดังนี้ (James et al., 2023):

1. **เรียบง่ายแต่มีประสิทธิภาพ:** มักให้ผลลัพธ์ที่ดีแม้กับข้อมูลที่ตัวแปรมีความสัมพันธ์เชิงเส้นกับ Target
2. **ตีความได้ง่าย:** สัมประสิทธิ์ (Coefficients) บอกได้โดยตรงว่าตัวแปรใดส่งผลอย่างไร
3. **เชื่อมจาก Regression:** ใช้หลักการจาก Linear Regression (บทที่ 8) มาดัดแปลงให้ทำ Classification ได้
4. **ให้ Probability:** ผลลัพธ์เป็นค่าความน่าจะเป็น (0-1) ไม่ใช่แค่ Class Label

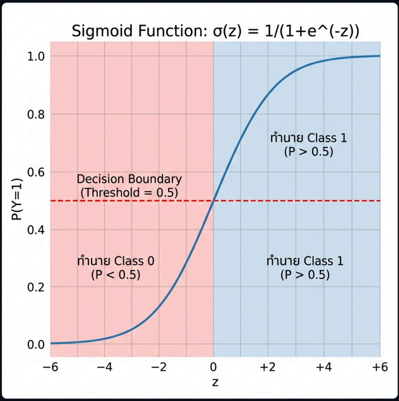
หลักการ: จาก Linear Regression สู่ Classification

ปัญหาหลักของการนำ Linear Regression มาทำ Classification คือ ผลลัพธ์เป็นค่าต่อเนื่อง (อาจติดลบหรือมากกว่า 1) ในขณะที่เราต้องการค่าความน่าจะเป็นของแต่ละ Class (0 ถึง 1)

ทางแก้: ใช้ **Sigmoid Function** (หรือ Logistic Function) แปลงค่าผลลัพธ์จาก Linear Regression ให้อยู่ในช่วง [0, 1]:

$$P(Y = 1|X) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

โดยที่ $z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + ... + \beta_p X_p$ (สมการ Linear Regression ปกติ)



Sigmoid Function

ตัวอย่าง: ทำนายการอนุมัติสินเชื่อ

สมมติว่าธนาคารพาณิชย์ในประเทศไทยต้องการสร้างโมเดลพิจารณาอนุมัติสินเชื่อส่วนบุคคล โดยพิจารณาจากข้อมูลรายได้ ประวัติเครดิต และอัตราส่วนหนี้สินต่อรายได้

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix

# 1. สร้างชุดข้อมูลจำลอง: อนุมัติสินเชื่อ (1 = อนุมัติ, 0 = ไม่อนุมัติ)
X, y = make_classification(n_samples=1000, n_features=5, n_informative=3,
                           n_redundant=1, random_state=42, flip_y=0.05)
feature_names = ['รายได้', 'Credit Score', 'อัตราหนี้/รายได้', 'อายุงาน', 'วงเงินที่ขอ']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. ปรับสเกลข้อมูล (สำคัญสำหรับ Logistic Regression)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 3. สร้างโมเดล Logistic Regression
log_reg = LogisticRegression(random_state=42, max_iter=1000)
log_reg.fit(X_train_scaled, y_train)

# 4. ทำนายและประเมินผล
y_pred = log_reg.predict(X_test_scaled)
y_prob = log_reg.predict_proba(X_test_scaled)  # ค่าความน่าจะเป็น

print("=== Logistic Regression: อนุมัติสินเชื่อ ===")
print(classification_report(y_test, y_pred, target_names=['ไม่อนุมัติ', 'อนุมัติ']))

# 5. ตีความ Coefficients
print("--- การตีความ Coefficients ---")
for name, coef in zip(feature_names, log_reg.coef_[0]):
    direction = "เพิ่มโอกาสอนุมัติ" if coef > 0 else "ลดโอกาสอนุมัติ"
    print(f"  {name:20s}: {coef:+.4f} ({direction})")

# 6. ตัวอย่างการทำนายพร้อม Probability
sample = X_test_scaled[0:1]
prob = log_reg.predict_proba(sample)[0]
print(f"ตัวอย่างลูกค้า:")
print(f"  P(ไม่อนุมัติ) = {prob[0]:.2%}")
print(f"  P(อนุมัติ)    = {prob[1]:.2%}")
print(f"  ผลตัดสิน: {'อนุมัติ' if prob[1] > 0.5 else 'ไม่อนุมัติ'}")
```

ผลลัพธ์:

```
=== Logistic Regression: อนุมัติสินเชื่อ ===
              precision    recall  f1-score   support

   ไม่อนุมัติ       0.88      0.91      0.90       101
     อนุมัติ       0.91      0.88      0.89        99

    accuracy                           0.90       200
   macro avg       0.90      0.89      0.89       200
weighted avg       0.90      0.90      0.89       200


--- การตีความ Coefficients ---
  รายได้              : +0.3065 (เพิ่มโอกาสอนุมัติ)
  Credit Score        : +1.1710 (เพิ่มโอกาสอนุมัติ)
  อัตราหนี้/รายได้    : -0.0047 (ลดโอกาสอนุมัติ)
  อายุงาน             : +0.2185 (เพิ่มโอกาสอนุมัติ)
  วงเงินที่ขอ         : +3.1420 (เพิ่มโอกาสอนุมัติ)

ตัวอย่างลูกค้า:
  P(ไม่อนุมัติ) = 65.03%
  P(อนุมัติ)    = 34.97%
  ผลตัดสิน: ไม่อนุมัติ
```

จุดเด่นของ Logistic Regression: การตีความ Coefficients

ข้อได้เปรียบสำคัญของ Logistic Regression เหนือ Decision Tree และ KNN คือ สัมประสิทธิ์บอก **ทิศทาง** และ **ขนาดผลกระทบ** ของแต่ละตัวแปรได้ชัดเจน:

Coefficient (β)	ความหมาย
$\beta > 0$ (บวก)	ตัวแปรนี้ส่งผลเพิ่มความน่าจะเป็นของ Class 1
$\beta < 0$ (ลบ)	ตัวแปรนี้ส่งผลลดความน่าจะเป็นของ Class 1
$\|\beta\|$ มาก	ตัวแปรนี้มีอิทธิพลสูง
$\|\beta\| \approx 0$	ตัวแปรนี้แทบไม่มีผล

ตัวอย่างการตีความในบริบทสินเชื่อ: “Credit Score มีค่า Coefficient = +1.25 หมายความว่า เมื่อ Credit Score เพิ่มขึ้น 1 หน่วย (หลัง Standardize) โอกาสอนุมัติสินเชื่อเพิ่มขึ้น” — ซึ่งเป็นข้อมูลที่ผู้บริหารธนาคารสามารถนำไปตัดสินใจได้ทันที

เปรียบเทียบ Logistic Regression กับอัลกอริทึมอื่น

คุณลักษณะ	Logistic Regression	Decision Tree	k-NN
Decision Boundary	เส้นตรง (Linear)	เส้นตรงแบบขั้นบันได	เส้นโค้งตามข้อมูล
Interpretability	สูง (Coefficients)	สูง (Rules)	ต่ำ (Black-box)
Probability Output	ใช่ (Calibrated)	ใช่ (แต่ไม่ค่อย Calibrated)	ใช่ (จากสัดส่วนเพื่อนบ้าน)
จัดการ Non-linearity	ไม่ได้ (ต้องทำ Feature Engineering)	ได้โดยธรรมชาติ	ได้โดยธรรมชาติ
ความเร็วฝึกสอน	เร็วมาก	เร็ว	เร็ว (แต่ทำนายช้า)
เหมาะกับ	Baseline, ข้อมูล Linearly Separable	ต้องการ Rules ที่อธิบายได้	ข้อมูลกลุ่มชัดเจน

สรุปประเด็นสำคัญ

- **Logistic Regression** ใช้ Sigmoid Function แปลงผลลัพธ์ Linear Regression ให้เป็นค่าความน่าจะเป็น [0, 1] เพื่อทำ Classification
- เป็น **Baseline Model** ที่ควรทดลองก่อนเสมอ เนื่องจากเรียบง่าย เร็ว และตีความได้ง่าย
- **Coefficients** บอกทิศทางและขนาดผลกระทบของแต่ละตัวแปร ทำให้สื่อสารกับ Stakeholders ได้ดี
- ข้อจำกัดคือต้องการ **Linear Decision Boundary** — หากข้อมูลมีความสัมพันธ์ซับซ้อน ควรใช้ Decision Tree หรือ Ensemble Methods (บทที่ 10)

คำถามท้าย Section

1. เหตุใดจึงไม่สามารถใช้ Linear Regression โดยตรงมาทำ Classification ได้ จงยกตัวอย่างปัญหาที่จะเกิดขึ้นหากใช้
2. ในระบบคัดกรองผู้ป่วย COVID-19 ของโรงพยาบาลในจังหวัดขอนแก่น หาก Logistic Regression ให้ค่า P(ติดเชื้อ) = 0.75 สำหรับผู้ป่วยรายหนึ่ง แพทย์ควรตั้ง Threshold ไว้ที่เท่าใด (0.5 หรือ 0.3)? จงอธิบายเหตุผลโดยอ้างอิงความสัมพันธ์กับ Precision-Recall Tradeoff
3. หากผู้บริหารธนาคารต้องการทราบว่าตัวแปรใดมีผลต่อการอนุมัติสินเชื่อมากที่สุด ระหว่าง Logistic Regression กับ k-NN อัลกอริทึมใดที่เหมาะสมกว่า? จงอธิบายเหตุผล

สรุปท้ายบท

ในบทนี้ นักศึกษาได้เรียนรู้เกี่ยวกับ: - **Classification:** ใช้ทำนายหมวดหมู่ — คำถามแบบ ‘อะไร?’ หรือ ‘ใช่/ไม่ใช่?’ ใช้ Classification - **Decision Tree:** อธิบายการตัดสินใจด้วยกฎ If-Then ตีความได้ง่าย แต่เสี่ยง Overfitting - **k-NN:** ทำนายจากเพื่อนบ้านที่ใกล้ที่สุด — ต้อง Scale ข้อมูลก่อนใช้ - **Logistic Regression:** Baseline Model สำคัญที่สุด — ใช้ Sigmoid แปลง Linear เป็น Probability - **Metrics:** Precision, Recall, F1-Score เลือกใช้ตามบริบท — Accuracy ไม่เหมาะกับ Imbalanced Data

คำศัพท์สำคัญ: เมื่อใดที่ควรใช้ปัญหา Classification, ประเภทของปัญหา Classification, ตัวอย่างการประยุกต์ใช้ในบริบทประเทศไทย, การสร้างโมเดล Classification เบื้องต้นด้, เกณฑ์การแบ่งข้อมูล (Splitting Criteria), การสร้างแบบจำลองและการอ่านผล (Visualizat, ไฮเปอร์พารามิเตอร์ (Hyperparameters) และ, ข้อดีและข้อเสียของ Decision Tree

---

เอกสารอ้างอิง

- Grus, J. (2019). *Data Science from Scratch* (2nd ed.). O’Reilly.
- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python* (2nd ed.). Springer.
- Knaflic, C. N. (2015). *Storytelling with Data: A Data Visualization Guide for Business Professionals*. Wiley.
- McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O’Reilly Media.
- VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O’Reilly Media.
- Wolpert, D. H. (1996). The Lack of A Priori Distinctions Between Learning Algorithms. *Neural Computation*, 8(7), 1341-1390.

✏ แบบฝึกหัดท้ายบท

☐ ส่วนที่ 1: คำถามทบทวน

คำชี้แจง: *จงเลือกคำตอบที่ถูกต้องที่สุดเพียงข้อเดียว*

1 สถานการณ์ใดต่อไปนี้เหมาะสมกับการใช้โมเดล Classification มากที่สุด

A การพยากรณ์ยอดขายสินค้าของร้านค้าปลีกในเดือนถัดไป

B การทำนายว่านักศึกษาจะได้รับเกรดเฉลี่ยสะสม (GPAX) เท่าใดเมื่อสำเร็จการศึกษา

C การพิจารณาอนุมัติสินเชื่อของธนาคาร (อนุมัติ หรือ ไม่อนุมัติ)

D การจัดกลุ่มลูกค้าของห้างสรรพสินค้าที่มีพฤติกรรมการซื้อคล้ายคลึงกันโดยไม่มีป้ายกำกับล่วงหน้า

▶ แสดงเฉลย

2 ในอัลกอริทึม Decision Tree เกณฑ์ใดที่นิยมใช้ในการเลือก Feature สำหรับแตกกิ่ง (Splitting) เพื่อให้ได้โหนดที่มีความบริสุทธิ์ (Purity) มากที่สุด

A Euclidean Distance

B Gini Impurity หรือ Information Gain

C Mean Squared Error (MSE)

D K-Means

▶ แสดงเฉลย

3 พิจารณาอัลกอริทึม k-Nearest Neighbors (k-NN) หากกำหนดค่า k = 1 จะส่งผลต่อโมเดลอย่างไร

A โมเดลจะมีอคติสูง (High Bias) และเกิด Underfitting

B โมเดลจะมีความแปรปรวนสูง (High Variance) และเสี่ยงต่อการเกิด Overfitting

C โมเดลจะใช้เวลาในการคำนวณน้อยลงอย่างมากเมื่อเทียบกับ k = 100

D โมเดลจะไม่สามารถจัดการกับข้อมูลที่มีหลายมิติ (High-dimensional data) ได้

▶ แสดงเฉลย

4 ระบบคัดกรองผู้ป่วยโรคมะเร็งของโรงพยาบาลแห่งหนึ่ง ต้องการลดโอกาสที่จะวินิจฉัยผู้ที่เป็นโรคมะเร็งผิดพลาดว่า “ไม่เป็นโรค” (False Negative) ให้เหลือน้อยที่สุด ในกรณีนี้ ควรให้ความสำคัญกับ Classification Metric ข้อใดมากที่สุด

A Precision

B Recall (Sensitivity)

C Specificity

D Accuracy

▶ แสดงเฉลย

5 ข้อใดกล่าวถูกต้องเกี่ยวกับการเปรียบเทียบและเลือกโมเดล Classification

A โมเดลที่มีค่า Accuracy สูงสุดบน Training Set คือโมเดลที่ดีที่สุดเสมอ

B k-NN เป็นโมเดลที่สามารถอธิบายเหตุผลในการตัดสินใจ (Interpretability) ได้ดีกว่า Decision Tree

C หากชุดข้อมูลมีปัญหา Imbalanced Data ควรพิจารณาใช้ F1-Score หรือ Area Under the ROC Curve (AUC-RO

C แทน Accuracy

D Decision Tree ไม่จำเป็นต้องทำการปรับจูน Hyperparameter (เช่น Max Depth) เนื่องจากโมเดลสามารถเรียนรู้ได้เองอย่างสมบูรณ์

▶ แสดงเฉลย

☐ ส่วนที่ 2: คำถามอัตนัย

คำชี้แจง: *จงอธิบาย วิเคราะห์ หรือเปรียบเทียบตามประเด็นที่กำหนดให้ชัดเจน*

1 การเลือกใช้โมเดลระหว่าง Decision Tree และ k-Nearest Neighbors (k-NN)

สมมติว่านักศึกษาได้รับมอบหมายให้สร้างระบบปัญญาประดิษฐ์เพื่อช่วยแพทย์วินิจฉัยโรคเบาหวาน (เป็น/ไม่เป็น) จากข้อมูลผลตรวจเลือด แพทย์ต้องการทราบว่า “ทำไม” ระบบจึงตัดสินใจว่าผู้ป่วยรายนี้มีความเสี่ยง นักศึกษาจะเลือกใช้อัลกอริทึมใดระหว่าง Decision Tree และ k-NN จงอธิบายเหตุผลประกอบ พร้อมระบุข้อควรระวังของอัลกอริทึมที่เลือก

▶ แสดงแนวคำตอบ

2 ปัญหา Imbalanced Data และการประเมินประสิทธิภาพ

บริษัท E-commerce รายหนึ่งในประเทศไทยมีข้อมูลการทำธุรกรรม 100,000 รายการ โดยเป็นธุรกรรมปกติ 99,000 รายการ และธุรกรรมฉ้อโกง (Fraud) 1,000 รายการ หากวิศวกรข้อมูลสร้างโมเดลที่ทำนายว่า “ทุกธุรกรรมเป็นธุรกรรมปกติ” โมเดลนี้จะมีค่า Accuracy เท่าใด? และจงอธิบายว่าเหตุใด Accuracy จึงไม่ใช่มาตรวัดที่เหมาะสมในกรณีนี้ ควรใช้มาตรวัดใดแทน?

▶ แสดงแนวคำตอบ

3 ผลกระทบของการทำ Data Scaling ต่อโมเดล k-NN

หากชุดข้อมูลประกอบด้วย Feature 2 ตัว คือ “อายุ” (มีค่า 18-60 ปี) และ “รายได้ต่อปี” (มีค่า 150,000 - 1,200,000 บาท) การนำข้อมูลนี้ไปฝึกสอนโมเดล k-NN โดยตรงโดยไม่ผ่านกระบวนการ Data Scaling จะเกิดผลกระทบอย่างไร จงอธิบายหลักการทางคณิตศาสตร์ที่อยู่เบื้องหลังผลกระทบนี้

▶ แสดงแนวคำตอบ

ส่วนที่ 3: แบบฝึกหัด Coding

คำชี้แจง: *จงเขียนโปรแกรมภาษา Python เพื่อแก้ปัญหาตามที่โจทย์กำหนด โดยใช้ไลบรารี* `scikit-learn` *และ* `pandas`

ข้อ 1. การสร้างและประเมินผล Decision Tree สำหรับทำนายการพ้นสภาพนักศึกษา โจทย์: กำหนดชุดข้อมูลจำลองของนักศึกษามหาวิทยาลัย (ข้อมูลถูกสร้างขึ้นในโค้ด) ประกอบด้วยเกรดเฉลี่ย (GPA), อัตราการเข้าเรียน (Attendance_Rate), และระยะทางจากที่พักถึงมหาวิทยาลัย (Distance_km) จงเขียนโปรแกรมเพื่อ: 1. แบ่งข้อมูลเป็น Training Set (80%) และ Test Set (20%) 2. สร้างและฝึกสอนโมเดล `DecisionTreeClassifier` โดยกำหนด `max_depth=3` และ `random_state=42` 3. ทำนายผลบน Test Set 4. แสดงผลลัพธ์เป็น `classification_report` และ `confusion_matrix`

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.datasets import make_classification

# 1. สร้างชุดข้อมูลจำลอง (Synthetic Data)
# สมมติให้ Class 0 = ปกติ (Retained), Class 1 = พ้นสภาพ (Dropped Out)
X, y = make_classification(n_samples=500, n_features=3, n_informative=3,
                           n_redundant=0, weights=[0.8, 0.2], random_state=42)

# แปลงเป็น DataFrame เพื่อให้เห็นภาพชัดเจน
df = pd.DataFrame(X, columns=['GPA_Scaled', 'Attendance_Scaled', 'Distance_Scaled'])
df['Status'] = y

# 2. แบ่งข้อมูลเป็น Training Set และ Test Set (80/20)
X_data = df.drop('Status', axis=1)
y_data = df['Status']
X_train, X_test, y_train, y_test = train_test_split(X_data, y_data, test_size=0.2, random_state=42, str

# 3. สร้างและฝึกสอนโมเดล Decision Tree
# กำหนด max_depth=3 เพื่อป้องกัน Overfitting และให้ง่ายต่อการอธิบาย
dt_model = DecisionTreeClassifier(max_depth=3, random_state=42)
dt_model.fit(X_train, y_train)

# 4. ทำนายผลบน Test Set
y_pred = dt_model.predict(X_test)

# 5. ประเมินประสิทธิภาพโมเดล
print("=== Confusion Matrix ===")
print(confusion_matrix(y_test, y_pred))
print("=== Classification Report ===")
# target_names ช่วยให้ report อ่านง่ายขึ้น
print(classification_report(y_test, y_pred, target_names=['Retained (0)', 'Dropped Out (1)']))
```

ผลลัพธ์:

```
=== Confusion Matrix ===
[[79  1]
 [ 2 18]]

=== Classification Report ===
                 precision    recall  f1-score   support

   Retained (0)       0.98      0.99      0.98        80
Dropped Out (1)       0.95      0.90      0.92        20

       accuracy                           0.97       100
      macro avg       0.96      0.94      0.95       100
   weighted avg       0.97      0.97      0.97       100
```

ข้อ 2. การปรับจูนค่า *k* ในโมเดล k-Nearest Neighbors (k-NN) โจทย์: ธนาคารแห่งหนึ่งต้องการจำแนกประเภทลูกค้าบัตรเครดิต (Premium และ Standard) จากพฤติกรรมการใช้จ่าย ข้อมูลนี้มีความแตกต่างของสเกล (Scale) จงเขียนโปรแกรมเพื่อ: 1. ทำ Data Scaling โดยใช้ `StandardScaler` 2. สร้างลูปเพื่อฝึกสอนโมเดล `KNeighborsClassifier` โดยทดสอบค่า *k* ตั้งแต่ 1 ถึง 15 (เฉพาะเลขคี่) 3. เก็บค่า Accuracy ของ Test Set สำหรับแต่ละค่า *k* 4. แสดงผลค่า *k* ที่ให้ Accuracy สูงที่สุด

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from sklearn.datasets import make_blobs

# 1. สร้างชุดข้อมูลจำลองพฤติกรรมลูกค้าบัตรเครดิต
# จำนวนธุรกรรมต่อเดือน และ ยอดใช้จ่ายเฉลี่ย (สเกลต่างกันมาก)
X, y = make_blobs(n_samples=300, centers=2, cluster_std=2.5, random_state=42)
# คูณค่าให้สเกลต่างกันจำลองความเป็นจริง
X[:, 0] = X[:, 0] * 10      # จำนวนธุรกรรม (หลักสิบ)
X[:, 1] = X[:, 1] * 10000   # ยอดใช้จ่าย (หลักหมื่น)

# 2. แบ่งข้อมูล Training และ Test Set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# 3. ทำ Data Scaling (สำคัญมากสำหรับ k-NN)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
# ข้อควรระวัง: ต้องใช้ parameters จากชุด Train ไปแปลงชุด Test (transform อย่างเดียว)
X_test_scaled = scaler.transform(X_test)

# 4. ทดสอบค่า k หลายๆ ค่าเพื่อหาโมเดลที่ดีที่สุด
k_values = range(1, 16, 2) # [1, 3, 5, 7, 9, 11, 13, 15]
best_k = 1
best_accuracy = 0
accuracy_results = []

print("=== ผลการทดสอบค่า k สำหรับโมเดล k-NN ===")
for k in k_values:
    # สร้างและฝึกสอนโมเดล
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)

    # ทำนายและวัดผล
    y_pred = knn.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)
    accuracy_results.append(acc)

    print(f"k = {k:2d} | Accuracy = {acc:.4f}")

    # เก็บค่า k ที่ดีที่สุด
    if acc > best_accuracy:
        best_accuracy = acc
        best_k = k
```

```
print("-" * 40)
print(f"สรุป: ค่า k ที่เหมาะสมที่สุดคือ k = {best_k} ด้วยความแม่นยำ (Accuracy) = {best_accuracy:.4f}")
```

ผลลัพธ์:

```
=== ผลการทดสอบค่า k สำหรับโมเดล k-NN ===
k =  1 | Accuracy = 0.9667
k =  3 | Accuracy = 0.9667
k =  5 | Accuracy = 0.9667
k =  7 | Accuracy = 0.9556
k =  9 | Accuracy = 0.9556
k = 11 | Accuracy = 0.9556
k = 13 | Accuracy = 0.9667
k = 15 | Accuracy = 0.9556
----------------------------------------
สรุป: ค่า k ที่เหมาะสมที่สุดคือ k = 1 ด้วยความแม่นยำ (Accuracy) = 0.9667
```

← บทที่ 8: Regression — การทำนายค่าต่อเนื่

บทที่ 10: Ensemble Methods →